Geometric Algebra (GA)

Daniel Hug

2025

1 Documentation

Overview (high-level initial intros on youtube):

- *https://www.thestrangeloop.com*
- *https://www.youtube.com/@sudgylacmoe*
- *https://www.youtube.com/@bivector*

Sources:

- https://projectivegeometricalgebra.org
- https://terathon.com/blog/poor-foundations-ga.html
- https://bivector.net
- https://www.youtube.com/@bivector
- https://www.youtube.com/@sudgylacmoe
- https://www.youtube.com/watch?v=ItGlUbFBFfc
- https://www.youtube.com/watch?v=v-WG02ILMXA&t=1476s

1.1 Introduction

The ga library is intended to make Geometric Algebra (GA) more accessible by providing functionality for numerical calculations required for simulation of physical systems.

It implements functionality for two- and three-dimensional spaces with Euclidean geometry. Currently, it provides data types and operations to handle the algebras of Euclidean geometric algebra ($ega2d, ega3d$) and projective Euclidean geometric algebra ($pga2dp, pga3dp$). The library efficiently handles fully populated multivectors, even and odd-grade multivectors as well as their components like scalars, vectors, bivectors, trivectors, and pseudoscalars of the corresponding linear spaces. It also provides operations to work with these objects. It was designed with extensibility in mind and can be extended to provide functionality for handling additional algebras, e.g. spacetime-algebra (STA), with limited effort.

The main library is a header only library in the `ga` sub-folder of the library which can be used by including either

```
#include ga/ga_ega.hpp // and/or
#include ga/ga_pga.hpp
```

and by making the corresponding namespaces accessible by stating

```
using namespace hd::ga;      // and
using namespace hd::ga::ega; // either/or
using namespace hd::ga::pga;
```

in your application code. For a simple usage example please have a look either at `ga/ga_test/src/ga_ega_test.cpp` or at `ga/ga_test/src/ga_pga_test.cpp`.

At the heart of the library there are user defined types that model mathematical objects of the respective geometric algebra. They are realized using type tagging in order to maximize reuse and to avoid code duplication. All types are defined in `ga/detail/type_t`.

To give an example for $EGA2D$, there are type aliases defined in the file `ga/detail/type_t/type2d.hpp` which provide types for scalars, vectors, pseudoscalars, even grade multivectors and full multivectors with the corresponding number of components. We have to distinguish between scalars and pseudoscalars at type level in order to allow for complement operations

(defined later on) that can distinguish those types for each algebra. Following types are provided:

```
namespace hd::ga {

template <typename T> using Scalar2d
                            = Scalar_t<T,scalar2d_tag>;
template <typename T> using Vec2d = Vec2_t<T,vec2d_tag>;
template <typename T> using PScalar2d
                            = Scalar_t<T,pscalar2d_tag>;
template <typename T> using MVec2d_E = MVec2_t<T,mvec2d_e_tag>;
template <typename T> using MVec2d = MVec4_t<T,mvec2d_tag>;

} // namespace hd::ga
```

Based on these template aliases types like `Vec2d<T>` can be used directly. However, for increased user convenience additional template specializations have been defined in `ga/ga_usr_types.hpp` as

```
using scalar2d = Scalar2d<value_t>;
using vec2d = Vec2d<value_t>;
using pscalar2d = PScalar2d<value_t>;
using mvec2d_e = MVec2d_E<value_t>;
using mvec2d = MVec2d<value_t>;
```

where `value_t` is defined for the whole library in `ga/value_t.hpp`. Thus the user can simply select `value_t=double` (set as default value) or `float` for the library and the user application at one location. There is no need to use template parameters at all, if the user-defined types like `scalar2,vec2d, pscalar2d,mvec2d_e`, and `mvec2d` are used for the application.

All algebras follow the same pattern. Just to give a second example for $PGA3DP$ following user types are defined: `scalar3dp,vec3dp,bivec3dp, trivec3dp,pscalar3dp,mvec3dp_e,mvec3dp_u,` and `mvec3dp`, where we additionally have defined types for bivectors, trivectors and uneven grade multivectors, as required to implement the algebra.

To work with the user defined algebraic objects geometric algebra operations using these types are defined in the ga library. You will find all operations that are required to work with linear vector spaces and operations specific to geometric algebra.

All product expressions for inner, outer, interior and geometric products as well as their regressive counterparts are generated by programs in the subfolder `ga/ga_prdxpr`. User input to configure product generation is provided in the header files of the corresponding algebra as well. The generated coefficient expressions are used in the source code to implement dot products, wedge products, commutator products, contractions, expansions, geometric products or their regressive counterparts, as well as sandwich products describing rotors and motors of the corresponding algebra. If certain products are required for your applicaton, but not yet implemented in the ga library, they can be easily generated within `ga/ga_prdxpr/src_prdxpr` as needed and added with minimal effort.

As mentioned before, we distinguish inner, interior, outer and geometric products.

- Inner ($a \cdot b$) and interior products, like contractions and expansions ($a \vee b^\star$, $a \wedge b^\star$), are grade reducing operations. The inner product combines arguments of the same type and grade, while the interior products involve duals of one of their arguments, combining and linking arguments of different grades. Inner products are linked to the metric of the modelled space.

- The outer product ($a \wedge b$) is a grade-increasing operation adding the grades of its arguments. Outer products, like the wedge product, connect subspaces represented by their operands and create new subspaces defined by the span of the subspaces of the operands. They establish a join-operation between the involved subspaces.

- The geometric product ($a \wedge\!\!\!\wedge b$) is the combination of both types of products in one expression. They can result in multivectors consisting of parts with different grades as result of the operation. The main advantage of the geometric product is its invertibility under certain circumstances.

- Regressive products ($a \circ b \equiv \underline{\overline{a} \cdot \overline{b}}$, $a \vee b \equiv \underline{\overline{a} \wedge \overline{b}}$, $rcmt(a,b) \equiv \underline{cmt(\overline{a},\overline{b})}$, $a \vee\!\!\!\vee b \equiv \underline{\overline{a} \wedge\!\!\!\wedge \overline{b}}$) use complement operations on their arguments before and after applying the operation[1]. The regressive wedge product creates an intersection of the (sub-)spaces used by their arguments, thus effectively creating a meet-operation between the involved spaces.

[1] for definition of the left and right complement operations marked with underbar and overbar, see next section

- Sandwich products ($R \wedge\!\!\!\wedge u \wedge\!\!\!\wedge \widetilde{R}$ for EGA, $M \vee\!\!\!\vee u \vee\!\!\!\vee \underset{\sim}{M}$ for PGA) are used to implement orthogonal transformations of geometric objects. They realize rotations for EGA and general motions including rotation and translation for PGA by concatenating two consecutive reflections, used as primitives to realize the resulting transformations. This can be achieved algebraically by multiplying the object symmetrically from the left- and right-hand side with the appropriate unitized objects, we either call rotors R in EGA or motors M in PGA.

Following bi-argument operations $op(a,b) = a\ op\ b$ and their regressive variants $rop(a,b) = \overline{a\ op\ \overline{b}}$ are implemented (with `op(a,b)` or `rop(a,b)` as the name of the implemented function). Using A vs. a implies $gr(A) \geq gr(a)$, where $gr(arg)$ returns the grade of the argument arg:

- Inner product, delivering a scalar: $a \cdot b$, `dot(a,b)`
- Outer product of a j- and a k-vector, delivering a $j+k$-vector in $span(a,b)$: $a \wedge b$, `wdg(a,b)`
- Geometric product: $a \wedge\!\!\!\wedge b$, `operator*(a,b)`
  - For vectors a and b: $a \wedge\!\!\!\wedge b = a \cdot b + a \wedge b$
  - For a vector a and a multivector B: $a \wedge\!\!\!\wedge B = B \gg a + a \wedge B$
  - For a multivector A and a vector b: $A \wedge\!\!\!\wedge b = b \ll A + A \wedge b$
- Commutator product (the asymmetric part of the geometric product): $cmt(a,b) \equiv \frac{1}{2}(a \wedge\!\!\!\wedge b - b \wedge\!\!\!\wedge a)$,
- Regressive commutator product (the asymmetric part of the regressive geometric product):
  $rcmt(a,b) \equiv \frac{1}{2}(\underline{\overline{a} \wedge\!\!\!\wedge \overline{b} - \overline{b} \wedge\!\!\!\wedge \overline{a}}) = \frac{1}{2}(a \vee\!\!\!\vee b - b \vee\!\!\!\vee a)$, `rcmt(a,b)`
- Regressive inner product, delivering a pseudoscalar: $a \circ b$, `rdot(a,b)` (for comparison of the dot product and the regressive dot product for *pga3dp* see table 1 at the end of section 1.2)
- Regressive outer product: $a \vee b$, `rwdg(a,b)`
- Regressive geometric product: $a \vee\!\!\!\vee b$, `rgpr(a,b)`
- Left (bulk) contraction: $a \rfloor B = a \ll B = a_\star \vee B$, `operator<<(a,B)`, which implements `left_bulk_contract(a,B)`
- Right (bulk) contraction: $B \lfloor a = B \gg a = B \vee a^\star$, `operator>>(B,a)`, which implements `right_bulk_contract(B,a)`

- Left weight contraction: $a_\star \vee B$, `left_weight_contract(a,B)`
- Right weight contraction: $B \vee a^\star$, `right_weight_contract(B,a)`
- Left bulk expansion: $A_\star \wedge b$, `left_bulk_expand(A,b)`
- Right bulk expansion: $a \wedge B^\star$, `right_bulk_expand(a,B)`
- Left weight expansion: $A_\star \wedge b$, `left_weight_expand(A,b)`
- Right bulk expansion: $a \wedge B^\star$, `right_weight_expand(a,B)`

where the items from the left contraction downwards are all considered interior products. They operate directly on the subspaces occupied by the objects. The product operation is performed after applying a dualization operation to one operand.

The product tables for all products are generated in `ga/ga_prdxpr`, and coefficient expressions for specific argument types like scalars, vector, bivectors, etc. can be generated based on those product tables using configuration options of `ga/ga_prdxpr`.

PGA explicitly means Projective Geometric Algebra in the context of the ga library. There is also an alternative approach called Plane-based Geometric Algebra (also abbreviated *PGA*) that is based on a space that is dual to the space modelled and used here. The alternative approach was introduced by [Gunn, 2020] and further promoted for applications in projective geometry by [Dorst and De Keninck, 2022]. That model uses dual representations of objects represented in the algebra (it uses $G^*(3,0,1)$, a space dual to $G(3,0,1)$). Planes are used instead of vectors to model everything, e.g. a point is modelled by three planes intersecting in a point. Both representations are mathematically equivalent, i.e. they are dual to each other. They are linked by the definition of complements and the interconnection between geometric and regressive geometric product, as defined above: $a \vee b \equiv \underline{\overline{a} \wedge \overline{b}}$. In the plane-based approach the geometric product is used to operate on complements and duals, which are defined as the primary objects used for the approach, while the approach introduced by Lengyel in [Lengyel, 2024b] favors the use of points, vectors, bivectors and planes plus the explicit use of regressive products and complements as well as duals for applying geometric operations to these objects. The latter approach unifies the use of the wedge product for join operations and of the regressive wedge product for meet operations for application in *EGA* and *PGA*. For our work we use $G(3,0,1)$.

1.2 Basics

The following is provided for Euclidean geometric algebra of two-dimensional space (*EGA2D*) using an orthormal basis $\boldsymbol{e}_1$, $\boldsymbol{e}_2$:

$$(\boldsymbol{e}_1)^2 = \boldsymbol{e}_1 \wedge \boldsymbol{e}_1 = \boldsymbol{e}_1 \cdot \boldsymbol{e}_1 = \boldsymbol{e}_1^2 = +1, \text{ and } \boldsymbol{e}_2^2 = +1$$

$$s_{2d} = s\,\mathbf{1} \tag{1a}$$

$$v_{2d} = v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 \tag{1b}$$

$$ps_{2d} = ps\,\boldsymbol{e}_{12} = ps\,\mathbb{1} \tag{1c}$$

equation (1a) is the scalar part s_{2d}, equation (1b) contains the vector part v_{2d}, and equation (1c) contains the pseudoscalar part ps_{2d}. The index $2d$ is typically omitted when clear from context. The four basis elements are $\{\mathbf{1}, \boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_{12}\}$. Using this basis a multivector M of $2d$-space is defined as

$$M = s\,\mathbf{1} + v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + ps\,\boldsymbol{e}_{12} \tag{2}$$

where equation (2) contains three parts: the scalar part $s\,\mathbf{1}$ (basis element is the scalar $\mathbf{1}$; if $\mathbf{1}$ is not shown, it is implicitly assumed for scalar values), the vector part v (basis elements $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$) and the pseudoscalar part $ps\,\boldsymbol{e}_{12} = ps\,\mathbb{1}$ (basis element is $I_{2d} = \boldsymbol{e}_{12}$, which is sometimes written as $\mathbb{1}$ to show its character as pseudoscalar of this space. I_{2d} is a bivector in two-dimensional Euclidean space).

For Euclidean geometric algebra of three-dimensional space (*EGA3D*) using an orthonormal basis $\boldsymbol{e}_1$, $\boldsymbol{e}_2$, $\boldsymbol{e}_3$ there are:

$$(\boldsymbol{e}_1)^2 = \boldsymbol{e}_1 \wedge \boldsymbol{e}_1 = \boldsymbol{e}_1 \cdot \boldsymbol{e}_1 = \boldsymbol{e}_1^2 = +1, \boldsymbol{e}_2^2 = +1, \text{ and } \boldsymbol{e}_3^2 = +1$$

$$s_{3d} = s\,\mathbf{1} \tag{3a}$$

$$v_{3d} = v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + v_3\boldsymbol{e}_3 \tag{3b}$$

$$B_{3d} = b_{23}\boldsymbol{e}_{23} + b_{31}\boldsymbol{e}_{31} + b_{12}\boldsymbol{e}_{12} \tag{3c}$$

$$ps_{3d} = ps\,\boldsymbol{e}_{123} = ps\,\mathbb{1} \tag{3d}$$

equation (3a) is the scalar part s_{3d}, equation (3b) is the vector part v_{3d}, equation (3c) is the bivector part B_{3d}, and equation (3d) is the pseudoscalar part ps_{3d}. The index $3d$ is typically omitted when clear from context. Compared to the $2d$-case it is obvious, that all parts depend on context, specifically on the dimensionality of the modelled space, and thus need to be defined and treated accordingly.[2] The basis elements are $\{\mathbf{1}, \boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3, \boldsymbol{e}_{23}, \boldsymbol{e}_{31}, \boldsymbol{e}_{12}, \boldsymbol{e}_{123}\}$.

[2] *hint*: Since C++ is a statically typed language, those types need to be well-defined and distinguishable from each other. This is especially important for the different scalar and pseudoscalar types. The ga library uses type tagging extensively for that purpose.

Using this basis a multivector M of $3d$-space is defined as

$$M = s\,\mathbf{1} + v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + v_3\boldsymbol{e}_3 + b_{23}\boldsymbol{e}_{23} + b_{31}\boldsymbol{e}_{31} + b_{12}\boldsymbol{e}_{12} + ps\,\boldsymbol{e}_{123} \tag{4}$$

where equation (4) contains four parts: the scalar component $s\,\mathbf{1}$ (basis element is the scalar $\mathbf{1}$), the vector part v (basis elements $\boldsymbol{e}_1$, $\boldsymbol{e}_2$ and $\boldsymbol{e}_3$), the bivector part B (basis elements $\boldsymbol{e}_{23}$, $\boldsymbol{e}_{31}$ and $\boldsymbol{e}_{12}$) and the pseudoscalar part $ps\,\boldsymbol{e}_{123} = ps\,\mathbb{1}$ (basis element is $I_{3d} = \boldsymbol{e}_{123}$, which is sometimes written as $\mathbb{1}$ to show its character as pseudoscalar of this space. I_{3d} is a trivector in three-dimensional Euclidean space).

Projective geometric algebra of two-dimensional Euclidean space (*pga2dp*) is modelled by embedding it in a three-dimensional space. Using an orthonormal basis $\boldsymbol{e}_1$, $\boldsymbol{e}_2$, $\boldsymbol{e}_3$ there are following basis vectors:

$$(\boldsymbol{e}_1)^2 = \boldsymbol{e}_1 \wedge \boldsymbol{e}_1 = \boldsymbol{e}_1 \cdot \boldsymbol{e}_1 = \boldsymbol{e}_1^2 = +1, \boldsymbol{e}_2^2 = +1, \text{ and } \boldsymbol{e}_3^2 = 0$$

$$s_{2dp} = s\,\mathbf{1} \tag{5a}$$

$$v_{2dp} = v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + v_3\boldsymbol{e}_3 \tag{5b}$$

$$B_{2dp} = b_{23}\boldsymbol{e}_{23} + b_{31}\boldsymbol{e}_{31} + b_{12}\boldsymbol{e}_{12} \tag{5c}$$

$$ps_{2dp} = ps\,\boldsymbol{e}_{321} = ps\,\mathbb{1} \tag{5d}$$

The most obvious difference is the basis vector $\boldsymbol{e}_3$ that squares to zero and the pseudoscalar of that space that is chosen to be $\boldsymbol{e}_{321}$. The trivector $\boldsymbol{e}_{321}$ squares to 0, because of it's component $\boldsymbol{e}_3$ contained as a factor. Equation (5a) is the scalar part s_{2dp}, equation (5b) is the vector part v_{2dp}, equation (5c) is the bivector part B_{2dp}, and equation (5d) is the pseudoscalar part ps_{2dp}. The index $2dp$ is typically omitted when clear from context. The eight basis elements are $\{\mathbf{1}, \boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3, \boldsymbol{e}_{23}, \boldsymbol{e}_{31}, \boldsymbol{e}_{12}, \boldsymbol{e}_{321}\}$. Using this basis a multivector M of two-dimensional projective space ($2dp$) is defined as

$$M = s\,\mathbf{1} + v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + v_3\boldsymbol{e}_3 + b_1\boldsymbol{e}_{23} + b_2\boldsymbol{e}_{31} + b_3\boldsymbol{e}_{12} + ps\,\boldsymbol{e}_{321} \tag{6}$$

where equation (6) contains four parts: the scalar component $s\,\mathbf{1}$ (basis element is the scalar $\mathbf{1}$), the vector part v (basis elements $\boldsymbol{e}_1$, $\boldsymbol{e}_2$ and $\boldsymbol{e}_3$), the bivector part B (basis elements $\boldsymbol{e}_{23}$, $\boldsymbol{e}_{31}$ and $\boldsymbol{e}_{12}$) and the pseudoscalar part $ps\,\boldsymbol{e}_{321} = ps\,\mathbb{1}$ (basis element is $I_{2dp} = \boldsymbol{e}_{321}$, which is sometimes written as $\mathbb{1}$ to show its character as pseudoscalar of this space. It is a trivector in two-dimensional Euclidean projective space that squares to zero).

Projective geometric algebra of three-dimensional Euclidean space (*pga3dp*) is modelled by embedding it in a four-dimensional space. Using an orthonormal

basis $\boldsymbol{e}_1$, $\boldsymbol{e}_2$, $\boldsymbol{e}_3$, $\boldsymbol{e}_4$ there are following basis vectors:

$$(\boldsymbol{e}_1)^2 = \boldsymbol{e}_1 \wedge \boldsymbol{e}_1 = \boldsymbol{e}_1 \cdot \boldsymbol{e}_1 = \boldsymbol{e}_1^2 = +1, \boldsymbol{e}_2^2 = +1, \boldsymbol{e}_3^2 = +1, \text{ and } \boldsymbol{e}_4^2 = 0$$

$$s_{3dp} = s\,\mathbf{1} \tag{7a}$$

$$v_{3dp} = v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + v_3\boldsymbol{e}_3 + v_4\boldsymbol{e}_4 \tag{7b}$$

$$B_{3dp} = b_{41}\boldsymbol{e}_{41} + b_{42}\boldsymbol{e}_{42} + b_{43}\boldsymbol{e}_{43} + b_{23}\boldsymbol{e}_{23} + b_{31}\boldsymbol{e}_{31} + b_{12}\boldsymbol{e}_{12} \tag{7c}$$

$$t_{3dp} = t_{423}\boldsymbol{e}_{423} + t_{431}\boldsymbol{e}_{431} + t_{412}\boldsymbol{e}_{412} + t_{321}\boldsymbol{e}_{321} \tag{7d}$$

$$ps_{3dp} = ps\,\boldsymbol{e}_{1234} = ps\,\mathbb{1} \tag{7e}$$

Now we have a basis vector $\boldsymbol{e}_4$ that squares to zero. The bivector has six components, and there is a trivector t. The pseudoscalar $\boldsymbol{e}_{1234}$ is a quadvector that squares to 0, because of it's component $\boldsymbol{e}_4$ contained as a factor. Equation (7a) is the scalar part s_{3dp}, equation (7b) is the vector part v_{3dp}, equation (7c) is the bivector part B_{3dp}, equation (7d) is the trivector part t_{3dp}, and equation (7e) is the pseudoscalar part ps_{3dp}. The index $3dp$ is typically omitted when clear from context. The sixteen basis elements of $pga3d$ are: $\{\mathbf{1}, \boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3, \boldsymbol{e}_4, \boldsymbol{e}_{41}, \boldsymbol{e}_{42}, \boldsymbol{e}_{43}, \boldsymbol{e}_{23}, \boldsymbol{e}_{31}, \boldsymbol{e}_{12}, \boldsymbol{e}_{423}, \boldsymbol{e}_{431}, \boldsymbol{e}_{412}, \boldsymbol{e}_{321}, \boldsymbol{e}_{1234}\}$. A multivector M of three-dimensional projective space ($3dp$) is defined as

$$\begin{aligned} M = {} & s\,\mathbf{1} \\ & + v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + v_3\boldsymbol{e}_3 + v_4\boldsymbol{e}_4 \\ & + b_{41}\boldsymbol{e}_{41} + b_{42}\boldsymbol{e}_{42} + b_{43}\boldsymbol{e}_{43} + b_{23}\boldsymbol{e}_{23} + b_{31}\boldsymbol{e}_{31} + b_{12}\boldsymbol{e}_{12} \\ & + t_{423}\boldsymbol{e}_{423} + t_{431}\boldsymbol{e}_{431} + t_{412}\boldsymbol{e}_{412} + t_{321}\boldsymbol{e}_{321} \\ & + ps\,\boldsymbol{e}_{1234} \end{aligned} \tag{8}$$

where equation (8) contains five parts: the scalar component $s\,\mathbf{1}$ (basis element is the scalar $\mathbf{1}$), the vector part v (basis elements $\boldsymbol{e}_1$, $\boldsymbol{e}_2$, $\boldsymbol{e}_3$, $\boldsymbol{e}_4$), the bivector part B (basis elements $\boldsymbol{e}_{41}$, $\boldsymbol{e}_{42}$, $\boldsymbol{e}_{43}$, $\boldsymbol{e}_{23}$, $\boldsymbol{e}_{31}$, $\boldsymbol{e}_{12}$), the trivector part t (basis elements $\boldsymbol{e}_{423}$, $\boldsymbol{e}_{431}$, $\boldsymbol{e}_{412}$, $\boldsymbol{e}_{321}$) and the pseudoscalar part $ps\,\boldsymbol{e}_{1234} = ps\,\mathbb{1}$ (basis element is $I_{3dp} = \boldsymbol{e}_{1234}$, which is sometimes written as $\mathbb{1}$ to show its character as pseudoscalar of this space. It is a quadvector in three-dimensional Euclidean projective space that squares to zero).

It is important to note that it is seldomly required to use a full multivector in the respective algebra. For most applications it is sufficient to use a blade or k-vector exclusively, which represents one grade of the multivector, e.g. a vector or a bivector. This reduces the effort in terms of memory and computational requirements significantly – typically reduced to a level that is required to to solve the underlying application problem anyway.

The complement operation used in the library is defined in [Lengyel, 2024b]. It is uniquely determined with respect to the outer product (not with respect to the geometric product, as commonly used for dualization by many autors working with GA). This is essential to generate consistent signs of expressions, operators, complements and duals. The left complement $\underline{u}$ and the right complement $\overline{u}$ are defined as

$$\underline{u} \wedge u = I_n \tag{9a}$$
$$u \wedge \overline{u} = I_n \tag{9b}$$

with $I_n = \boldsymbol{e}_1 \wedge \boldsymbol{e}_2 \wedge \ldots \wedge \boldsymbol{e}_n$ as the pseudoscalar of the n-dimensional algebra, and the wedge symbol ($\wedge$) as the operator symbol of the outer product. The effect of the complement is to turn basis elements not contained in the object u into the constituting basis elements of the complement of the object $\underline{u}$ or $\overline{u}$. Connecting the object and its complement with the wedge products creates an object that fills the available space in the sense of requiring all basis elements of the space to represent it. The full space itself is represented by the pseudoscalar of that space, since the pseudoscalar I_n is formed by the product of all n basis vectors of the space. An implication of the definition is that the complement ($\underline{u}$ or $\overline{u}$) is orthogonal to u. The complement is defined as given above in equation (9) for any basis element. It is extended to full multivectors A and B by linearity as

$$\overline{sA} = s\overline{A}, \quad \overline{A+B} = \overline{A} + \overline{B} \tag{10a}$$
$$\underline{sA} = s\underline{A}, \quad \underline{A+B} = \underline{A} + \underline{B} \tag{10b}$$

where s is a scalar. The left and right complements fulfill

$$\overline{\underline{u}} = u \tag{11}$$

thus the left and right complement operations are inverses to each other. This is valid independent of the sequence of application, independent of the grade of the argument a, and independent of the dimension of the space modelled in the algebra. In spaces of odd dimension, where we only have one complement, applying the complement twice of course returns the original object as well.

Based on this it is possible to define so-called regressive products using rules analog to deMorgan's rules in Boolean algebra as follows (definitions can be found in [Lengyel, 2024b] and in [Browne, 2012]):

$$a \vee b \equiv \underline{\overline{a} \wedge \overline{b}} \tag{12a}$$
$$a \vee b \equiv \overline{\underline{a} \wedge \underline{b}} \tag{12b}$$

The same complement is applied to both arguments first, the resulting expressions are combined with the wedge product, and the inverse complement operation is used to transform the outcome into the final result, the regressive product. The inverted wedge symbol ($\vee$) is used for the regressive wedge product. Both definitions (12a) and (12b) are equivalent. Equation (12a) is used to derive expressions for regressive products in `ga/ga_prdxpr`.

There are some additional formulas for complements which are useful for working with GA expressions and thus are provided here for reference. Taking the right and left complements of (12a) and (12b) we end up with

$$\overline{a \vee b} = \overline{\overline{a} \wedge \overline{b}} = \overline{a} \wedge \overline{b} \tag{13a}$$

$$\underline{a \vee b} = \underline{\underline{a} \wedge \underline{b}} = \underline{a} \wedge \underline{b} \tag{13b}$$

Substituting $a = \underline{u}$ and $b = \underline{v}$ into (13a) and $a = \overline{u}$ and $b = \overline{v}$ into (13b) and exchanging the left and right hand sides of the equations we end up with

$$u \wedge v = \overline{\underline{u} \vee \underline{v}} \tag{14a}$$

$$u \wedge v = \underline{\overline{u} \vee \overline{v}} \tag{14b}$$

which shows that the outer and the regressive outer product can be expressed by each other.

The complement operation is used to define a unique dualization operation that works for cases with non-degenerate as well as for cases with degenerate metric, like in projective geometric algebra PGA. This is the main advantage compared to the usual prodedure of dualization by multiplying the object with the pseudoscalar (or its inverse) using the geometric product, since the latter breaks down for spaces with degenerate metrics. For an overview on all complements in $pga3dp$ see table 2 at the end of this section.

The metric is defined by the dot product of the basis vectors as

$$\mathfrak{g}_{ij} \equiv \boldsymbol{e}_i \cdot \boldsymbol{e}_j \tag{15}$$

It is a square matrix that is the identity matrix I_n for EGA, since all basis vectors square to $\boldsymbol{e}_i \cdot \boldsymbol{e}_i = +1$, and $\boldsymbol{e}_i \cdot \boldsymbol{e}_j = 0$ for $i \neq j$. It is invertible in this case. However, for PGA one basis vector squares to zero, and thus there is one zero on the main diagonal of the corresponding matrix of the metric, which is therefore not invertible. It is called degenerate for that reason.

The extended metric $\mathbf{G}$ is a so-called exomorphism matrix that fulfills

$$\mathbf{G}(A \wedge B) = \mathbf{G}(A) \wedge \mathbf{G}(B) \tag{16}$$

for any multivector A and B (see [Lengyel, 2024b], pp. 61ff.).
Regarding $\mathbf{G}(u) = \mathbf{G}u$ as a unary operation on u we can define the corresponding regressive operation by first taking the complement of the argument and then applying the opposed complement on the result. This defines the regressive metric $\mathbb{G}$ (named anti-metric by Lengyel). It is defined as

$$\mathbb{G}u \equiv \underline{\mathbf{G}\overline{u}} = \overline{\mathbf{G}\underline{u}} \tag{17}$$

$\mathbf{G}$ and $\mathbb{G}$ are related by

$$\mathbf{G}\mathbb{G} = det(\mathfrak{g})I_n \tag{18}$$

Thus, if all basis vectors square to ± 1, we have $\mathbf{G}\mathbb{G} = \pm I_n$, and both are invertible. However, if one basis vector squares to zero, we get $\mathbf{G}\mathbb{G} = 0$, which means that none of them is invertible, i.e. we have a degenerate metric.

For $\mathbb{G}$ there is the relation

$$\mathbb{G}(A \vee B) = \mathbb{G}(A) \vee \mathbb{G}(B) \tag{19}$$

that is complementary to equation (16).

In PGA the product $\mathbf{G}u$ defines the bulk of of an object u. It is defined as

$$u_\bullet = \mathbf{G}u \tag{20}$$

The bulk only contains components that *do not* contain the projective dimension ($\boldsymbol{e}_3$ for $pga2dp$ or $\boldsymbol{e}_4$ for $pga3dp$) as a factor. It contains positional information on u relative to the origin. Zero bulk means that u contains the origin.

In PGA the product $\mathbb{G}u$ defines the weight of of an object u. It is defined as

$$u_\circ = \mathbb{G}u \tag{21}$$

The weight selects components that *do* contain the projective dimension ($\boldsymbol{e}_3$ for $pga2dp$ or $\boldsymbol{e}_4$ for $pga3dp$) as a factor. It contains directional information on u, i.e. its attitude and direction independent of its position. Zero weight means that u is contained in the horizon, i.e. it is infinitely far away.

This effective split for each object into bulk and weight in projective algebras can be written as

$$u = u_\bullet + u_\circ \tag{22}$$

This split is also relevant for the corresponding norms

$$\|u\| = \|u\|_\bullet + \|u\|_\circ = \sqrt{A \cdot A} + \sqrt{A \circ A} \tag{23}$$

where the left hand side is called the geometric norm and the right hand side consists of the sums of the bulk and weight norms. The bulk norm returns a scalar value of the corresponding algebra, whereas the weight norm provides a pseudoscalar value of the respective algebra. Consequently, the geometric norm is an object consisting of two distinct parts, defined here as a dual number in the corresponding algebra. When we apply unitization to geometric objects we devide the object in such a way that the magnitude of the weight norm becomes a value of one. Both, bulk and weight norms, have a positive argument under the square root due to the definition of the inner product and the regressive inner product.

Using the metric and the right complement, the right bulk dual $A^\star$ is defined as

$$A^\star = \overline{\mathbf{G}A} = \overline{A_\bullet} \tag{24}$$

where A is an arbitrary multivector, $\mathbf{G}$ the extended metric and $\mathbf{G}A$ a matrix-vector-product between them. $A^\star$ is also called the Hodge dual. The multiplication with the metric happens before the application of the complement operation.

There is also a corresponding left bulk dual $A_\star$ operation using the metric and the left complement. It is defined as

$$A_\star = \underline{\mathbf{G}A} = \underline{A_\bullet} \tag{25}$$

The Hodge star (symbolized by a filled star $\star$) is used as dualization operator in both cases. The lower or upper position symbolizes the left and right application. The difference between the hodge dual and the complement is in both cases that multiplying the multivector with the extended metric happens *before* taking the corresponding complement. However, this does not lead to different results in spaces where the metric is the identity matrix. It only yields different results in spaces with a more involved metric, e.g. like in projective spaces with a non-invertible (degenerate) metric as we have in *PGA*.

The effect of applying the metric before taking the complement is equivalent to applying the complement operation on the bulk part of the object u exclusively, and not on the full object u. Of course, the dualization can be applied to the weight part as well. This operation (symbolized by an unfilled star "$\star$") uses the regressive extended metric and is defined for the right weight dual $A^\star$ using the right complement as

$$A^\star = \overline{\mathbb{G}A} = \overline{A_\circ} \tag{26}$$

for an arbitrary multivector A. The corresponding left weight dual $A_\star$ is defined as

$$A_\star = \underline{\mathbb{G}A} = \underline{A_\circ} \tag{27}$$

using the left complement on the weight part of A.

Relations to the geometric product and the regressive geometric product are given by

$$A^\star = \widetilde{A} \wedge I_n = \widetilde{A} \wedge \mathbb{1} \tag{28a}$$
$$A_\star = I_n \wedge \widetilde{A} = \mathbb{1} \wedge \widetilde{A} \tag{28b}$$
$$A^\star = \underset{\sim}{A} \vee \mathbf{1} \tag{28c}$$
$$A_\star = \mathbf{1} \vee \underset{\sim}{A} \tag{28d}$$

with $I_n = \boldsymbol{e}_1 \wedge \boldsymbol{e}_2 \wedge \ldots \wedge \boldsymbol{e}_n = \mathbb{1}$ as the pseudoscalar of the n-dimensional space. $\widetilde{A}$ is the reversion operation, and $\underset{\sim}{A}$ is the regressive reversion operation applied to A (e.g. with $A = \boldsymbol{e}_1 \wedge \boldsymbol{e}_2$ the reverse is $\widetilde{A} = \boldsymbol{e}_2 \wedge \boldsymbol{e}_1$), and "$\wedge$" and "$\vee$" are the symbols for the geometric and regressive geometric products. As the complement, the reverse is defined for any basis element. It is extended to full multivectors A and B by linearity as

$$\widetilde{sA} = s\widetilde{A}, \quad \widetilde{A+B} = \widetilde{A} + \widetilde{B}, \quad \widetilde{A \wedge B} = \widetilde{B} \wedge \widetilde{A} \tag{29a}$$
$$\underset{\sim}{sA} = s\underset{\sim}{A}, \quad \underset{\sim}{A+B} = \underset{\sim}{A} + \underset{\sim}{B}, \quad \underset{\sim}{A \vee B} = \underset{\sim}{B} \vee \underset{\sim}{A} \tag{29b}$$
$$\widetilde{\widetilde{A}} = A, \quad \text{and} \quad \underset{\sim}{\underset{\sim}{A}} = A \tag{29c}$$

where s is a scalar. The reverse and regressive reverse are their own inverses, i.e. applying the same reverse twice returns the original object.

For an overview on all bulk and weight duals in *pga3dp* see table 3 at the end of this section. The reverses can be found in table 4 at the end of the section as well.

Following identities link the expansions and contractions and thus the outer and inner products:

$$
\begin{aligned}
(A \wedge B)^{\star} &= A^{\star} \vee B^{\star} && \text{(30a)}\\
A \wedge B^{\star} &= (A \cdot B) \wedge \mathbb{1}, \quad \text{when } \texttt{gr(A)} = \texttt{gr(B)} && \text{(30b)}\\
A \wedge B^{\star} &= A \circ B, \quad \text{when } \texttt{gr(A)} = \texttt{gr(B)} && \text{(30c)}
\end{aligned}
$$

$$
\begin{aligned}
(A \vee B)^{\star} &= A^{\star} \wedge B^{\star} && \text{(30d)}\\
A \vee B^{\star} &= A \cdot B, \quad \text{when } \texttt{gr(A)} = \texttt{gr(B)} && \text{(30e)}\\
A \vee B^{\star} &= (A \circ B) \vee \mathbf{1}, \quad \text{when } \texttt{gr(A)} = \texttt{gr(B)} && \text{(30f)}
\end{aligned}
$$

The influence of the metric for the dualization operation can be seen in its effect for the different algebras which also depends on the dimension of the modelled space:

- $EGA2D$: $A_{\star}$ = `left_dual(A)`, $A^{\star}$ = `right_dual(A)`
- $EGA3D$: $A_{\star}$ = `dual(A)`, $A^{\star}$ = `dual(A)`
- $PGA2DP$: $A_{\star}$ = `bulk_dual(A)`, $A^{\star}$ = `bulk_dual(A)`
- $PGA2DP$: $A_{\star}$ = `weight_dual(A)`, $A^{\star}$ = `weight_dual(A)`
- $PGA3DP$: $A_{\star}$ = `left_bulk_dual(A)`, $A^{\star}$ = `right_bulk_dual(A)`
- $PGA3DP$: $A_{\star}$ = `left_weight_dual(A)`, $A^{\star}$ = `right_weight_dual(A)`

In even-dimensional spaces ($EGA2D$, $PGA3DP$) we have to distinguish left and right operations, while in odd-dimensional spaces ($EGA3D$, $PGA2DP$) we don't, since they produce the same result. The "left" operations are typically used on the left-hand side of the binary operation and the "right" operations on the right-hand side of the product operator. The only deviation from that pattern occurs for the definition of the regressive operations, as defined in (12a) or (12b).

In a two-dimensional algebra, like $EGA2D$, the effect of the right dual of a vector u is to turn the vector by +90°, i.e. into the orientation of $\boldsymbol{e}_{12}$, while the left dual turns the vector by −90°, i.e. into the direction against the orientation of $\boldsymbol{e}_{12}$. This statement is valid for both right- and left-handed coordinate systems.

The multiplicative identity of the geometric product and the wedge product is the scalar $\mathbf{1}$:

$$\mathbf{1} \wedge\!\!\!\wedge a = a \wedge\!\!\!\wedge \mathbf{1} = a \tag{31a}$$
$$\mathbf{1} \wedge a = a \wedge \mathbf{1} = a \tag{31b}$$

The multiplicative identity of the regressive geometric product and the regressive wedge product is the pseudoscalar $\mathbb{1} = I_n$ of the respective algebra:

$$\mathbb{1} \vee\!\!\!\vee a = a \vee\!\!\!\vee \mathbb{1} = a \tag{32a}$$
$$\mathbb{1} \vee a = a \vee \mathbb{1} = a \tag{32b}$$

The geometric product combines the dimensions that are present in an object representation, e.g. for $pga2dp$:

$$\begin{aligned} \boldsymbol{e}_1 \wedge\!\!\!\wedge \boldsymbol{e}_1 &= 1 \\ \boldsymbol{e}_2 \wedge\!\!\!\wedge \boldsymbol{e}_2 &= 1 \\ \boldsymbol{e}_3 \wedge\!\!\!\wedge \boldsymbol{e}_3 &= 0 \end{aligned} \tag{33}$$

The regressive geometric product combines the dimensions that are <u>*not*</u> present in an object representation, e.g. for $pga2dp$:

$$\begin{aligned} \overline{\boldsymbol{e}_1} \vee\!\!\!\vee \overline{\boldsymbol{e}_1} &= \mathbb{1} \\ \overline{\boldsymbol{e}_2} \vee\!\!\!\vee \overline{\boldsymbol{e}_2} &= \mathbb{1} \\ \overline{\boldsymbol{e}_3} \vee\!\!\!\vee \overline{\boldsymbol{e}_3} &= 0 \end{aligned} \tag{34}$$

where eqs. (34) can be shown to be equivalent to eqs. (33) by taking the left complement of both sides and applying the complement definition (12a) to it.

The inner product between multivectors A and B is defined as

$$A \cdot B = A^T \mathbf{G} B \tag{35}$$

with $\mathbf{G}$ as the extended metric and matrix multiplication on the right hand side. It satisfies the identity

$$A \cdot B = \langle A \wedge\!\!\!\wedge \tilde{B} \rangle_0 = \langle B \wedge\!\!\!\wedge \tilde{A} \rangle_0 \tag{36}$$

where $\wedge\!\!\!\wedge$ stands for the geometric product within the grade projection operator $\langle\rangle_k$ for grade k.

Using the various usually used notations in literature the contraction is explicitly defined as

$$A \rfloor B = A \ll B = A_\star \vee B \tag{37a}$$
$$B \lfloor A = B \gg A = B \vee A^\star \tag{37b}$$

with $\star$ denoting the Hodge dual which is formed by the respective complement operation (in spaces of even dimension the left or right complement respectively, and in spaces of odd dimension the complement function regardless of the side of the operand). For blades A and B, they satisfy

$$A \rfloor B = A \ll B = \langle B \wedge \tilde{A} \rangle_{gr(B)-gr(A)} \tag{38a}$$
$$B \lfloor A = B \gg A = \langle \tilde{A} \wedge B \rangle_{gr(B)-gr(A)} \tag{38b}$$

and fulfill the requirement that $A \rfloor B = A \lfloor B = A \cdot B$ whenever A and B have the same grade. In this case the contractions reduce to the inner product.

The geometric product between a vector v and a blade B is defined in terms of interior and exterior products, i.e. the right contraction "$\lfloor$", the left contraction "$\rfloor$" and wedge product "$\wedge$" as follows:

$$v \wedge B = B \lfloor v + v \wedge B \tag{39a}$$
$$B \wedge v = v \rfloor B + B \wedge v \tag{39b}$$

or sometimes expressed using the right shift "$\gg$" or left shift "$\ll$" operators for the contractions (as implemented in the ga library)

$$v \wedge B = B \gg v + v \wedge B \tag{40a}$$
$$B \wedge v = v \ll B + B \wedge v \tag{40b}$$

or sometimes by exclusively using the operations of the exterior algebra (wedge product "$\wedge$", bulk dual "$\star$" and regressive wedge product "$\vee$")

$$v \wedge B = B \vee v^\star + v \wedge B \tag{41a}$$
$$B \wedge v = v_\star \vee B + B \wedge v \tag{41b}$$

as an alternative. Each one is equivalent.

Be carefull, when using the equivalent formulas like the right-hand side of equations (41) when deriving regressive products. Resulting formulas may not be valid! The equivalent formulas for the regressive products must be derived directly from the regressive product table. To give one example, here

is the corresponding formula for the regressive geometric product for the case of a regressive product between a vector v and a bivector B:

$$v \vee B = -B_\star \wedge v + rcmt(v, B) \tag{42a}$$
$$B \vee v = -v \wedge B^\star + rcmt(B, v) \tag{42b}$$

It is obviously not the result one would get from creating the regressive versions directly from the right-hand side of equations (41). The reason is that the specific pre-conditions for the validity of the equivalence between the geometric product and the corresponding right-hand sides may change when taking the complements.

Every vector v can be decomposed into parts $v_\parallel$ parallel to a k-blade B and $v_\perp$ perpendicular to B, such that $v = v_\parallel + v_\perp$. For equation (40a): $B \gg v$ is a $(k-1)$-blade. If $v_\parallel \neq 0$ then $B \gg v$ represents a subspace of B. $v \wedge B$ is a $(k+1)$-blade. If $v_\perp \neq 0$ then $v \wedge B$ represents $span(v, B)$.
For arguments of equal grade the contractions reduce to the dot product, so that one can write specifically for two vectors a and b:

$$a \wedge\!\!\!\wedge b = a \cdot b + a \wedge b \tag{43}$$

[Lengyel, 2025] has shown that the geometric product can be derived by using operations from Grassmann algebra exclusively. These are the wedge product, the complement and the duality operations. This demonstrates that the geometric product is not a fundamental operation in its own right, but can be derived from more fundamental basic operations.

For that purpose Lengyel introduced the transwedge product of order k by following definition:

$$a \underset{k}{\wedge\!\!\!\wedge} b = \sum_{c \in \mathcal{B}_k} (\underline{c} \vee a) \wedge (b \vee c^\star) \tag{44}$$

where $\mathcal{B}_k$ is the set of all basis elements of order k. For $EGA3D$ it means for example $\mathcal{B}_0 = \{1\}$, $\mathcal{B}_1 = \{\boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3\}$, $\mathcal{B}_2 = \{\boldsymbol{e}_{23}, \boldsymbol{e}_{31}, \boldsymbol{e}_{12}\}$, and $\mathcal{B}_3 = \{\boldsymbol{e}_{123}\}$. From these expressions the geometric product can be formed by

$$a \wedge\!\!\!\wedge b = \sum_{k=0}^{n} (-1)^{k(k-1)/2} a \underset{k}{\wedge\!\!\!\wedge} b \tag{45}$$

For $k = 0$ the formula yields the outer product (wedge product). For higher values of k other products are generated which sum up to the full geometric product. Thus the transwedge products helps to decompose the geometric

product. The most interesting for applications are the wedge product resulting for $k = 0$ and the transwedge product of order $k = 1$.

The regressive wedge product in equation (44) can be replaced by the wedge product using the complement-based definitions given in equations (11) to (14b). That means for example that equation (44) can be transformed to the equivalent expression

$$a \underset{k}{\rightthreetimes} b = \sum_{c \in \mathcal{B}_k} (\underline{c \wedge \overline{a}}) \wedge (\underline{\overline{b} \wedge \overline{c^\star}}) \tag{46}$$

expressing it by using the wedge product exclusively.

Since every product can by assigned a corresponding regressive product in analogy to definition (12a) or (12b), the transwedge product (44) also has a regressive form:

$$a \underset{k}{\leftthreetimes} b \equiv \overline{\underline{a} \underset{k}{\rightthreetimes} \underline{b}} = \sum_{c \in \mathcal{B}_k} \overline{(\underline{c} \vee \underline{a}) \wedge (\underline{b} \vee c^\star)} \tag{47}$$

which in turn can be expressed in terms of the wedge product exclusively as:

$$a \underset{k}{\leftthreetimes} b = \sum_{c \in \mathcal{B}_k} \overline{(\underline{c \wedge a}) \wedge (\underline{b \wedge \overline{c^\star}})} \tag{48}$$

Equations (46) and (48) are used for calculation of the product tables of the (regressive) transwedge products of order k and the resulting (regressive) geometric products in `ga/ga_prdxpr`. The transwedge product is very useful to show that the geometric product can be derived from primitives of Grassmann algebra. When (regressive) transwedge product expressions are calculated, it can be shown that each transwedge product and the resulting coefficient expressions are equivalent to specific combinations of wedge-, commutator-, dot-products, and their regressive counterparts. This was done in `ga/src_prdxpr/ga_pga2dp_ops_products.hpp` for the (regressive) transwedge product of order $k = 1$ and as well in `ga/src_prdxpr/ga_pga3dp_ops_products.hpp` to show the equivalence.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_4$	$\boldsymbol{e}_{41}$	$\boldsymbol{e}_{42}$	$\boldsymbol{e}_{43}$	$\boldsymbol{e}_{23}$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	$\boldsymbol{e}_{321}$	$\mathbb{1}$
$\boldsymbol{u}\cdot\boldsymbol{u}$	$\mathbf{1}$	$\mathbf{1}$	$\mathbf{1}$	$\mathbf{1}$	0	0	0	0	$\mathbf{1}$	$\mathbf{1}$	$\mathbf{1}$	0	0	0	$\mathbf{1}$	0
$\boldsymbol{u}\circ\boldsymbol{u}$	0	0	0	0	$\mathbb{1}$	$\mathbb{1}$	$\mathbb{1}$	$\mathbb{1}$	0	0	0	$\mathbb{1}$	$\mathbb{1}$	$\mathbb{1}$	0	$\mathbb{1}$

Table 1: Dot and regressive dot products in pga3dp for every basis element.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_4$	$\boldsymbol{e}_{41}$	$\boldsymbol{e}_{42}$	$\boldsymbol{e}_{43}$	$\boldsymbol{e}_{23}$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	$\boldsymbol{e}_{321}$	$\mathbb{1}$
$\overline{\boldsymbol{u}}$	$\mathbb{1}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	$\boldsymbol{e}_{321}$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	$-\boldsymbol{e}_{41}$	$-\boldsymbol{e}_{42}$	$-\boldsymbol{e}_{43}$	$-\boldsymbol{e}_{1}$	$-\boldsymbol{e}_{2}$	$-\boldsymbol{e}_{3}$	$-\boldsymbol{e}_{4}$	$\mathbf{1}$
$\underline{\boldsymbol{u}}$	$\mathbb{1}$	$-\boldsymbol{e}_{423}$	$-\boldsymbol{e}_{431}$	$-\boldsymbol{e}_{412}$	$-\boldsymbol{e}_{321}$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	$-\boldsymbol{e}_{41}$	$-\boldsymbol{e}_{42}$	$-\boldsymbol{e}_{43}$	$\boldsymbol{e}_{1}$	$\boldsymbol{e}_{2}$	$\boldsymbol{e}_{3}$	$\boldsymbol{e}_{4}$	$\mathbf{1}$

Table 2: Right and left complements in pga3dp for every basis element.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_4$	$\boldsymbol{e}_{41}$	$\boldsymbol{e}_{42}$	$\boldsymbol{e}_{43}$	$\boldsymbol{e}_{23}$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	$\boldsymbol{e}_{321}$	$\mathbb{1}$
$\boldsymbol{u}^{\star}$	$\mathbb{1}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	0	0	0	0	$-\boldsymbol{e}_{41}$	$-\boldsymbol{e}_{42}$	$-\boldsymbol{e}_{43}$	0	0	0	$-\boldsymbol{e}_{4}$	0
$\boldsymbol{u}_{\star}$	$\mathbb{1}$	$-\boldsymbol{e}_{423}$	$-\boldsymbol{e}_{431}$	$-\boldsymbol{e}_{412}$	0	0	0	0	$-\boldsymbol{e}_{41}$	$-\boldsymbol{e}_{42}$	$-\boldsymbol{e}_{43}$	0	0	0	$\boldsymbol{e}_{4}$	0
$\boldsymbol{u}^{\star}$	0	0	0	0	$\boldsymbol{e}_{321}$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	0	0	0	$-\boldsymbol{e}_{1}$	$-\boldsymbol{e}_{2}$	$-\boldsymbol{e}_{3}$	0	$\mathbf{1}$
$\boldsymbol{u}_{\star}$	0	0	0	0	$-\boldsymbol{e}_{321}$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	0	0	0	$\boldsymbol{e}_{1}$	$\boldsymbol{e}_{2}$	$\boldsymbol{e}_{3}$	0	$\mathbf{1}$

Table 3: Right/left bulk duals and right/left weight duals in pga3dp for every basis element.

$\boldsymbol{u}$	$\mathbf{1}$	$\boldsymbol{e}_1$	$\boldsymbol{e}_2$	$\boldsymbol{e}_3$	$\boldsymbol{e}_4$	$\boldsymbol{e}_{41}$	$\boldsymbol{e}_{42}$	$\boldsymbol{e}_{43}$	$\boldsymbol{e}_{23}$	$\boldsymbol{e}_{31}$	$\boldsymbol{e}_{12}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	$\boldsymbol{e}_{321}$	$\mathbb{1}$
$\widetilde{\boldsymbol{u}}$	$\mathbf{1}$	$\boldsymbol{e}_{1}$	$\boldsymbol{e}_{2}$	$\boldsymbol{e}_{3}$	$\boldsymbol{e}_{4}$	$-\boldsymbol{e}_{41}$	$-\boldsymbol{e}_{42}$	$-\boldsymbol{e}_{43}$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	$-\boldsymbol{e}_{423}$	$-\boldsymbol{e}_{431}$	$-\boldsymbol{e}_{412}$	$-\boldsymbol{e}_{321}$	$\mathbb{1}$
$\underset{\sim}{\boldsymbol{u}}$	$\mathbf{1}$	$-\boldsymbol{e}_{1}$	$-\boldsymbol{e}_{2}$	$-\boldsymbol{e}_{3}$	$-\boldsymbol{e}_{4}$	$-\boldsymbol{e}_{41}$	$-\boldsymbol{e}_{42}$	$-\boldsymbol{e}_{43}$	$-\boldsymbol{e}_{23}$	$-\boldsymbol{e}_{31}$	$-\boldsymbol{e}_{12}$	$\boldsymbol{e}_{423}$	$\boldsymbol{e}_{431}$	$\boldsymbol{e}_{412}$	$\boldsymbol{e}_{321}$	$\mathbb{1}$

Table 4: Reverse and regressive reverse in pga3dp for every basis element.

1.3 Modelling motion

For translational motion in physical contexts we typically use the following naming convention for scalars or vectors:

$$\begin{aligned}
s(t) &= s \\
\dot{s}(t) &= v(t) = \dot{s} = v \\
\ddot{s}(t) &= \dot{v}(t) = a(t) = \ddot{s} = \dot{v} = a
\end{aligned}$$

while for rotational motion we typically use:

$$\begin{aligned}
\phi(t) &= \phi \\
\dot{\phi}(t) &= \omega(t) = \dot{\phi} = \omega \\
\ddot{\phi}(t) &= \dot{\omega}(t) = \ddot{\phi} = \dot{\omega}
\end{aligned}$$

Especially in PGA, we will be able to combine the translational and rotational aspects in one physical variable. Thus we will use a convention of using capital letters for physical quantities to distinguish them from the classical convention, especially so for bivectors and even-grade multivectors that do not occur in classical descriptions. We will use:

$B(t) = B$	(bivector)
$\dot{B}(t) = \Omega(t) = \dot{B} = \Omega$	(rate of change of bivector, bivector speed)
$\ddot{B}(t) = \dot{\Omega}(t) = \ddot{B} = \dot{\Omega}$	(rate of change of bivector speed)

especially when we want to emphazise rotational aspects of motion in PGA.

In order to model motion in PGA we need to work with the exponential function and apply it in the context of the geometric product (symbol "$\wedge$") and the regressive geometric product (symbol "$\vee$").

$$\exp(x) = \sum_{n=0}^{\infty} \frac{x^n}{n!} \qquad = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \frac{x^5}{5!} + \frac{x^6}{6!} + \frac{x^7}{7!} + \cdots \tag{49}$$

$$\cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n)!} \qquad = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \frac{x^8}{8!} - \frac{x^{10}}{10!} + \frac{x^{12}}{12!} - \frac{x^{14}}{14!} + \cdots \tag{50}$$

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!} \qquad = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} - \frac{x^{11}}{11!} + \frac{x^{13}}{13!} - \frac{x^{15}}{15!} + \cdots \tag{51}$$

If we substitute $x = i\phi$ with i as the imaginary unit that squares to -1, we get Euler's formula:

$$\begin{aligned}
\exp(i\phi) &= \sum_{n=0}^{\infty} \frac{(i\phi)^n}{n!} \\
&= 1 + i\phi + \frac{(i\phi)^2}{2!} + \frac{(i\phi)^3}{3!} + \frac{(i\phi)^4}{4!} + \frac{(i\phi)^5}{5!} + \frac{(i\phi)^6}{6!} + \frac{(i\phi)^7}{7!} + \cdots && (52) \\
&= 1 + i\phi - \frac{\phi^2}{2!} - i\frac{\phi^3}{3!} + \frac{\phi^4}{4!} + i\frac{\phi^5}{5!} - \frac{\phi^6}{6!} - i\frac{\phi^7}{7!} + \cdots \\
&= 1 - \frac{\phi^2}{2!} + \frac{\phi^4}{4!} - \frac{\phi^6}{6!} + \cdots + i(\phi - \frac{\phi^3}{3!} + \frac{\phi^5}{5!} - \frac{\phi^7}{7!} + \cdots) && (53)
\end{aligned}$$

$$\exp(i\phi) = \cos(\phi) + i\sin(\phi) \tag{54}$$

Applying this for geometric algebra we have to distinguish between calculating the powers in the Taylor series using the geometric product (in EGA) and using the regressive geometric product (in PGA). We assume a unitized bivector B and a scalar ϕ. Based on the geometric product we know that $B \wedge B = -\mathbf{1}$, and for its regressive counterpart we know that $B \vee B = -\mathbb{1}$. Thus, B plays a role equivalent to i in both cases. In addition, taking any value to the power of 0 results in the neutral element of the product, which is $\mathbf{1}$ for the geometric product, and $\mathbb{1}$ for the regressive geometric product. Taking the powers then works like this:

$$\begin{aligned}
&\cdots \\
\frac{(B\phi)^2_{\wedge}}{2!} &= \frac{1}{2!}(B\phi \wedge B\phi) = \frac{\phi^2}{2!}(B \wedge B) = \frac{\phi^2}{2!}(-\mathbf{1}) = -\frac{\phi^2}{2!} && (55a) \\
\frac{(B\phi)^3_{\wedge}}{3!} &= \frac{\phi^3}{3!}(B \wedge B \wedge B) = \frac{\phi^3}{3!}(-\mathbf{1} \wedge B) = -\frac{\phi^3}{3!}B && (55b) \\
&\cdots \\
\frac{(B\phi)^2_{\vee}}{2!} &= \frac{1}{2!}(B\phi \vee B\phi) = \frac{\phi^2}{2!}(B \vee B) = \frac{\phi^2}{2!}(-\mathbb{1}) = -\frac{\phi^2}{2!}\mathbb{1} && (55c) \\
\frac{(B\phi)^3_{\vee}}{3!} &= \frac{1}{3!}B\phi \vee B\phi \vee B\phi = \frac{1}{3!}\phi^3(-\mathbb{1} \vee B) = -\frac{\phi^3}{3!}B && (55d) \\
&\cdots
\end{aligned}$$

Thus, the result of exponentiating a bivector B is:

$$\begin{aligned}
\exp_{\wedge}(B\phi) &= \sum_{n=0}^{\infty} \frac{(B\phi)_{\wedge}^{n}}{n!} \\
&= 1 + B\phi + \frac{(B\phi)_{\wedge}^{2}}{2!} + \frac{(B\phi)_{\wedge}^{3}}{3!} + \frac{(B\phi)_{\wedge}^{4}}{4!} + \frac{(B\phi)_{\wedge}^{5}}{5!} + \frac{(B\phi)_{\wedge}^{6}}{6!} + \cdots \\
&= 1 + B\phi - \frac{\phi^2}{2!} - B\frac{\phi^3}{3!} + \frac{\phi^4}{4!} + B\frac{\phi^5}{5!} - \frac{\phi^6}{6!} - B\frac{\phi^7}{7!} + \cdots \\
&= 1 - \frac{\phi^2}{2!} + \frac{\phi^4}{4!} - \frac{\phi^6}{6!} + \cdots + B(\phi - \frac{\phi^3}{3!} + \frac{\phi^5}{5!} - \frac{\phi^7}{7!} + \cdots)
\end{aligned} \tag{56}$$

$$\exp_{\wedge}(B\phi) = \cos(\phi) + \sin(\phi)B \tag{57}$$

The last step of moving B after the brackets is allowed, since scalar factors commute with any object using the regular product. The result structurally is an even-grade multivector with a scalar and a bivector component. We get the correct sign and angle of rotation if we replace ϕ by $-\frac{1}{2}\phi$, which was not considered above to keep the formulas as simple as possible. The substitution yields the final form of a rotor R in EGA with B as unitized bivector and ϕ the rotation angle:

$$\begin{aligned}
R = \exp_{\wedge}(-\frac{\phi}{2}B) &= \cos(-\frac{\phi}{2}) + \sin(-\frac{\phi}{2})B \\
R &= \cos(\frac{\phi}{2}) - \sin(\frac{\phi}{2})B
\end{aligned} \tag{58}$$

For the case using the regressive geometric product it becomes:

$$\begin{aligned}
\exp_{\vee}(B\phi) &= \sum_{n=0}^{\infty} \frac{(B\phi)_{\vee}^{n}}{n!} \\
&= \mathbb{1} + B\phi + \frac{(B\phi)_{\vee}^{2}}{2!} + \frac{(B\phi)_{\vee}^{3}}{3!} + \frac{(B\phi)_{\vee}^{4}}{4!} + \frac{(B\phi)_{\vee}^{5}}{5!} + \frac{(B\phi)_{\vee}^{6}}{6!} + \cdots \\
&= \mathbb{1} + B\phi - \mathbb{1}\frac{\phi^2}{2!} - B\frac{\phi^3}{3!} + \mathbb{1}\frac{\phi^4}{4!} + B\frac{\phi^5}{5!} - \mathbb{1}\frac{\phi^6}{6!} - B\frac{\phi^7}{7!} + \cdots \\
&= \mathbb{1}(1 - \frac{\phi^2}{2!} + \frac{\phi^4}{4!} - \frac{\phi^6}{6!} + \cdots) + B(\phi - \frac{\phi^3}{3!} + \frac{\phi^5}{5!} - \frac{\phi^7}{7!} + \cdots)
\end{aligned} \tag{59}$$

$$\exp_{\vee}(B\phi) = \cos(\phi)\mathbb{1} + \sin(\phi)B \tag{60}$$

The last step of moving $\mathbb{1}$ and B after the brackets is allowed, since scalar factors, like the results of the trigonometric functions, commute with any object using the regular product. The result structurally is an even-grade multivector with a bivector and a pseudoscalar component.

Eqn. (57) allows us to model rotations in EGA, where the bivector B is the complement of the axis of rotation and $\theta = -2\phi$ is the angle of the rotation around that axis. If we directly want to specify the angle of rotation θ we need to substitute ϕ by $-\frac{\phi}{2}$ in eqn. (57). The rotated object X can then be obtained from the original object X_0 by:

$$X = R \wedge X_0 \wedge \widetilde{R} = \exp_{\wedge}(-\frac{\phi}{2}B) \wedge X_0 \wedge \exp_{\wedge}(\frac{\phi}{2}B) \quad \text{(for } EGA\text{)} \tag{61}$$

If we want to model translations and rotations in PGA, we need to look into another extension of the exponential function. We follow the approach defined by [Lengyel, 2024b] in chapter 3.2 and 3.6.2, where it is shown that a general motor in PGA can be expressed as a rotation around a unitized line l by an angle ϕ and a translation along l by a displacement δ as

$$\begin{aligned} M = \exp_{\vee}\left[(\frac{\delta}{2}\mathbf{1} + \frac{\phi}{2}\mathbb{1}) \vee l\right] &= \cos_{\vee}(\frac{\delta}{2}\mathbf{1} + \frac{\phi}{2}\mathbb{1}) + \sin_{\vee}(\frac{\delta}{2}\mathbf{1} + \frac{\phi}{2}\mathbb{1}) \vee \boldsymbol{l} \\ &= -\frac{\delta}{2}\mathbf{1}\sin(\frac{\phi}{2}) + \mathbb{1}\cos(\frac{\phi}{2}) + \left(\frac{\delta}{2}\mathbf{1}\cos(\frac{\phi}{2}) + \mathbb{1}\sin(\frac{\phi}{2})\right) \vee \boldsymbol{l} \\ M &= -\frac{\delta}{2}\mathbf{1}\sin(\frac{\phi}{2}) - \boldsymbol{l}^{\star}\frac{\delta}{2}\cos(\frac{\phi}{2}) + \boldsymbol{l}\sin(\frac{\phi}{2}) + \mathbb{1}\cos(\frac{\phi}{2}) \end{aligned} \tag{62}$$

where M structurally is an even grade multivector with scalar, bivector and pseudoscalar parts in $3D$ PGA. The argument in the exponential function again results in a bivector. This can be derived from inspecting the product table for the regressive geometric product. However, this time we only have a common factor of $\frac{1}{2}$, and not $-\frac{1}{2}$ as in EGA. This will impact some signs for the transformations in the regressive case for PGA, compared to the corresponding transformations in EGA. The resulting transformation is then obtained from

$$X = M \vee X_0 \vee \underset{\sim}{M} \quad \text{(for } PGA\text{)} \tag{63}$$

In order to dive deeper into modelling of motions we first start with the EGA case. It is identical to the derivation shown in [Dorst and De Keninck, 2023] for their plane-based approach. This will help to better recognize the differences to the PGA approach used here.

Before going into more detail, we need to prepare some formulas we later need for our derivations:

For EGA the rotor will now be written as M instead of R to make it easy to compare to our PGA-derivations using motors M later on. M as a rotor in EGA is an orthonomal transformation and thus satisfies

$$M \wedge \widetilde{M} = \widetilde{M} \wedge M = 1 \tag{64}$$

(remember: $M^{-1} = \widetilde{M}$), and therefore

$$\frac{d}{dt}(M \wedge \widetilde{M}) = \dot{M} \wedge \widetilde{M} + M \wedge \dot{\widetilde{M}} = 0 \tag{65}$$

It follows that $M \wedge \dot{\widetilde{M}} = -\dot{M} \wedge \widetilde{M}$, which results in

$$\dot{\widetilde{M}} = -\widetilde{M} \wedge \dot{M} \wedge \widetilde{M} \tag{66}$$

Starting from $\widetilde{M} \wedge M = 1$ instead yields the same result for $\dot{\widetilde{M}}$.

In order to model rigid body mechanics we consider two coordinate frames:

- A *world frame* (index w) that is normally chosen to be an inertial frame.
- A *body frame* (index b) that is attached to a rigid body. Typically at its center of gravity, but this is not strictly required. Rigid body's internal forces are in equilibrium and thus need not be considered explicitly. It is sufficient to describe the translational motion of the body's center of gravity and its rotational motion with respect to the center of gravity. The resulting motion of the body frame relative to the world frame then fully covers rigid body motion (kinematics and dynamics), which depends on distribution of the mass of the body as well as on external forces and moments acting on the body.
- To describe the motion we use a time dependend motor $M = M(t)$. At time $t = 0$ we call it $M_0 = M(t = 0)$. M_0 brings an element X of the body to its position X_0 at time $t = 0$.

World frame: From eqn. (58) we know the form of a general rotor in EGA and use the ansatz for a time dependent rotor $M(t)$ after the the object X was brought into its initial position by M_0 as $X_0 = M \wedge X \wedge \widetilde{M}$. If there is no difference between world and body frames at $t = 0$ then $M_0 = 1$:

$$M(t) = exp_{\wedge}(-\frac{1}{2}B_w(t)) \wedge M_0 \tag{67}$$

with $B_w(t)$ as a time-dependent bivector that models the rotational plane and the turning angle. The derivative of M with respect to time t is

$$\dot{M}(t) = -\frac{1}{2}\dot{B}_w(t) \wedge exp_{\wedge}(-\frac{1}{2}B_w(t)) \wedge M_0 = -\frac{1}{2}\dot{B}_w(t) \wedge M(t) \tag{68}$$

where we use that the exponential of the bivector and the bivector rate commute *(tbd: show why this is the case)*. Using $\Omega_w = \dot{B}_w(t)$ as the bivector rate of change in the *world frame*, we get

$$\dot{M} = -\frac{1}{2}\Omega_w \wedge M \tag{69}$$

By multiplying this from the right with "$\wedge \widetilde{M}$" this can transformed to

$$\Omega_w = -2\dot{M} \wedge \widetilde{M} \tag{70}$$

Body frame: Here we first move the object in the *body frame* and then transform it into the in the world frame by the transformation M_0. In this case eqn. (67) becomes

$$M(t) = M_0 \wedge exp_{\wedge}(-\frac{1}{2}B_b(t)) \tag{71}$$

which in turn restults in

$$\dot{M}(t) = -\frac{1}{2}M_0 \wedge exp_{\wedge}(-\frac{1}{2}B_b(t)) \wedge \dot{B}_b(t) = -\frac{1}{2}M(t) \wedge \dot{B}_b(t) \tag{72}$$

while eqn. (69) for the bivector rate of change in the *body frame* becomes

$$\dot{M} = -\frac{1}{2}M \wedge \Omega_b \tag{73}$$

using $\Omega_b = \dot{B}_b(t)$ as the bivector rate of change in the *body frame*. This can be transformed by multiplying it from the left by "$\widetilde{M} \wedge$" to obtain

$$\Omega_b = -2\widetilde{M} \wedge \dot{M} \tag{74}$$

From equations (69) and (73) we can see how world and body frame rates are connected to each other: $\Omega_w \wedge M = M \wedge \Omega_b$, which can be transformed into

$$\Omega_w = M \wedge \Omega_b \wedge \widetilde{M} \tag{75a}$$

$$\Omega_b = \widetilde{M} \wedge \Omega_w \wedge M \tag{75b}$$

Now we derive the rate of change of a point $X(t)$, i.e. its speed $\dot{X}(t)$, in the *world frame*, first assuming that $X(t=0) = X_0 = const.$, i.e. as a constant element that is transformed to become X by a time dependent motor $M(t)$. During the derivation we will use the associativity of the geometric product and the neutral element 1 of the geometric product. We can write

$$\begin{aligned}
\dot{X}(t) &= \frac{d}{dt}X(t) = \frac{d}{dt}(M \wedge X_0 \wedge \widetilde{M}) \\
&= \dot{M} \wedge X_0 \wedge \widetilde{M} + M \wedge X_0 \wedge \dot{\widetilde{M}} \\
&\overset{(66)}{=} \dot{M} \wedge 1 \wedge X_0 \wedge \widetilde{M} - M \wedge X_0 \wedge (\widetilde{M} \wedge \dot{M} \wedge \widetilde{M}) \\
&\overset{(64)}{=} \dot{M} \wedge (\widetilde{M} \wedge M) \wedge X_0 \wedge \widetilde{M} - (M \wedge X_0 \wedge \widetilde{M}) \wedge \dot{M} \wedge \widetilde{M} \\
&= (\dot{M} \wedge \widetilde{M}) \wedge (M \wedge X_0 \wedge \widetilde{M}) - (M \wedge X_0 \wedge \widetilde{M}) \wedge (\dot{M} \wedge \widetilde{M})
\end{aligned}$$

From eqn. (70) we can conclude that $-\frac{1}{2}\Omega_w(t) = \dot{M} \wedge \widetilde{M}$, resulting in

$$\begin{aligned}
\dot{X}(t) &= (-\frac{1}{2}\Omega_w(t)) \wedge X(t) - X(t) \wedge (-\frac{1}{2}\Omega_w(t)) \\
&= -\frac{1}{2}(\Omega_w \wedge X - X \wedge \Omega_w) \\
&= -cmt(\Omega_w, X) \\
\dot{X}_w &= cmt(X_w, \Omega_w)
\end{aligned} \tag{76}$$

with $cmt(X_w, \Omega_w)$ as the commutator product between X_w and Ω_w.

So, if the bivector rate of change Ω_w is known, the rate of change of X_w, which is $\dot{X}_w$, can be calculated from eqn. (76) using the commutator product.

It can be shown that $\dot{M} \wedge \widetilde{M}$ is always a bivector: From equations (70) and (65), and the linearity of the involved operations, we get

$$\Omega_w = -2\dot{M} \wedge \widetilde{M} = 2M \wedge \dot{\widetilde{M}} = 2(\widetilde{\dot{M} \wedge \widetilde{M}}) = -\widetilde{\Omega}_w \tag{77}$$

Considering the grade-dependent sign-reversal of the reverse in $3D$ EGA, Ω_w must either be a bivector or a trivector. Since $\dot{M}$ and $\widetilde{M}$ are even-grade multivectors, their geometric product results in an even-grade multivector as well, which does not have a trivector part. Thus, Ω_w and therefore $\dot{M} \wedge \widetilde{M}$ must indeed be a bivector.

This also assures, that the grade of X and $\dot{X}$ are identical. If X is a vector then $\dot{X}$ is a vector as well. This statement is true for any grade of X as long as Ω_w is a bivector.

If X_0 is not a constant element, but a function of time, which means that X_0 is a time dependent element in the *body frame* $X_0 = X_b(t)$, we have to add another term to the derivative in eqn. (76) as follows:

$$\begin{aligned}
\dot{X}(t) &= \frac{d}{dt}(M \wedge X_b \wedge \widetilde{M}) \\
&= \dot{M} \wedge X_b \wedge \widetilde{M} + M \wedge X_b \wedge \dot{\widetilde{M}} + M \wedge \dot{X}_b \wedge \widetilde{M} \\
&= cmt(M \wedge X_b \wedge \widetilde{M}, \Omega_w) + M \wedge \dot{X}_b \wedge \widetilde{M} \\
\dot{X}_w &= cmt(X_w, \Omega_w) + (\dot{X}_b)_w
\end{aligned} \tag{78}$$

Eqn. (78) is to be considered in the world frame. However, we can also do our derivations in a completely equivalent way using the corresponding bivector rates and rotors of the *body frame*.

Now we derive the rate of change of a point $X(t)$ in the *body frame*, again assuming first that $X(t=0) = X_0 = const.$. We can write

$$\begin{aligned}
\dot{X}(t) &= \dot{M} \wedge X_0 \wedge \widetilde{M} + M \wedge X_0 \wedge \dot{\widetilde{M}} \\
&\overset{(66)}{=} 1 \wedge \dot{M} \wedge 1 \wedge X_0 \wedge \widetilde{M} - M \wedge X_0 \wedge (\widetilde{M} \wedge \dot{M} \wedge \widetilde{M}) \\
&\overset{(64)}{=} M \wedge (\widetilde{M} \wedge \dot{M}) \wedge \widetilde{M} \wedge (M \wedge X_0 \wedge \widetilde{M}) - (M \wedge X_0 \wedge \widetilde{M}) \wedge 1 \wedge \dot{M} \wedge \widetilde{M} \\
&= \left(M \wedge (\widetilde{M} \wedge \dot{M}) \wedge \widetilde{M}\right) \wedge X(t) - X(t) \wedge \left(M \wedge (\widetilde{M} \wedge \dot{M}) \wedge \widetilde{M}\right) \\
&= \left(M \wedge (\widetilde{M} \wedge \dot{M}) \wedge \widetilde{M}\right) \wedge X(t) - X(t) \wedge \left(M \wedge (\widetilde{M} \wedge \dot{M}) \wedge \widetilde{M}\right)
\end{aligned}$$

From eqn. (74) we can conclude that $-\frac{1}{2}\Omega_b(t) = \widetilde{M} \wedge \dot{M}$, resulting in

$$\begin{aligned}
\dot{X}(t) &= \left(M \wedge (-\frac{1}{2}\Omega_b(t)) \wedge \widetilde{M}\right) \wedge X(t) - X(t) \wedge \left(M \wedge (-\frac{1}{2}\Omega_b(t)) \wedge \widetilde{M}\right) \\
&= -\frac{1}{2}\left((\Omega_b)_w \wedge X - X \wedge (\Omega_b)_w\right) \\
&= -\frac{1}{2}\left(\Omega_w \wedge X - X \wedge \Omega_w\right) \\
&= -cmt(\Omega_w, X) \\
\dot{X}_w &= cmt((X_b)_w, \Omega_w) = cmt(X_w, \Omega_w)
\end{aligned} \tag{79}$$

with $cmt(X_w, \Omega_w)$ as the commutator product between X_w and Ω_w. This is exactly the same result as before, just based on bivector rate measured in the *body frame*.

It can be shown that $\widetilde{M} \wedge \dot{M}$ is always a bivector with the same argument as before. So we don't repeat that here.

If X_0 is not a constant element, but itself a function of time $X_b(t)$, we have to add another term to the derivative in eqn. (79) again as follows:

$$\begin{aligned}
\dot{X}(t) &= \frac{d}{dt}(M \wedge X_b \wedge \widetilde{M}) \\
&= \dot{M} \wedge X_b \wedge \widetilde{M} + M \wedge X_b \wedge \dot{\widetilde{M}} + M \wedge \dot{X}_b \wedge \widetilde{M} \\
&= cmt(M \wedge X_b \wedge \widetilde{M}, \Omega_w) + M \wedge \dot{X}_b \wedge \widetilde{M} \\
&= cmt((X_b)_w, \Omega_w) + (\dot{X}_b)_w \\
\dot{X}_w &= cmt(X_w, \Omega_w) + (\dot{X}_b)_w \qquad (80)
\end{aligned}$$

Eqn. (80) is again to be considered in the *world frame*. The result derived in the *body frame* is of course identical to the result derived in the *world frame*.

In *EGA* rotation as a Euclidean isometry can be described with sandwich products based on the geometric product. Now looking into *PGA* Lengyel has shown in [Lengyel, 2024b] that Euclidean isometries (covering translations as well as rotations) can be described with sandwich products based on the regressive geometric product in *PGA*. If we do that, we can stay in the space $G(3,0,1)$ and do not need to switch to its dual space in order to implement these orthonormal transformations as already described above.

Now we repeat the derivation of the time-derivative $\dot{X}$ of a point $X(t)$ for the projective approach of *PGA* used here: We will use the regressive geometric product, $\mathbb{1}$ as its neutral element. The derivation is then completely analogous to the *EGA* case with the given substitutions.

M as a motor in *PGA* is an orthonomal transformation and satisfies

$$M \vee \underset{\sim}{M} = \underset{\sim}{M} \vee M = \mathbb{1} \qquad (81)$$

as the defining equation for a motor in *PGA* (remember: $M^{-1} = \underset{\sim}{M}$), leads to

$$\frac{d}{dt}(M \vee \underset{\sim}{M}) = \dot{M} \vee \underset{\sim}{M} + M \vee \dot{\underset{\sim}{M}} = 0 \qquad (82)$$

It follows that $M \vee \dot{\underset{\sim}{M}} = -\dot{M} \vee \underset{\sim}{M}$, which results in

$$\dot{\underset{\sim}{M}} = -\underset{\sim}{M} \vee \dot{M} \vee \underset{\sim}{M} \qquad (83)$$

Starting from $\underset{\sim}{M} \vee M = \mathbb{1}$ instead yields the same result for $\dot{\underset{\sim}{M}}$.

World frame: From eqn. (62) we know the form of a general motor in PGA and use the ansatz for a time dependent motor $M(t)$ after the the object X was brought into its initial position by M_0 as $X_0 = M \veebar X \veebar \underset{\sim}{M}$. If there is no difference between world and body frames at $t = 0$ then $M_0 = \mathbb{1}$:

$$M(t) = exp_{\veebar}(\frac{1}{2}B_w(t)) \veebar M_0 \tag{84}$$

with $B_w(t)$ as a time-dependent bivector that models the rotational plane and the turning angle. The derivative of M with respect to time t is

$$\dot{M}(t) = \frac{1}{2}\dot{B}_w(t) \veebar exp_{\veebar}(\frac{1}{2}B_w(t)) \veebar M_0 = \frac{1}{2}\dot{B}_w(t) \veebar M(t) \tag{85}$$

where we use that the exponential of the bivector and the bivector rate commute *(tbd: show why this is the case)*. Using $\Omega_w = \Omega_w(t) = \dot{B}_w(t)$ as the bivector rate of change in the *world frame*, we get

$$\dot{M} = \frac{1}{2}\Omega_w \veebar M \tag{86}$$

By multiplying this from the right with "$\veebar \underset{\sim}{M}$" this can transformed to

$$\Omega_w = 2\dot{M} \veebar \underset{\sim}{M} \tag{87}$$

Body frame: Here we first move the object in the *body frame* and then transform it into the in the world frame by the transformation M_0. In this case eqn. (84) becomes

$$M(t) = M_0 \veebar exp_{\veebar}(\frac{1}{2}B_b(t)) \tag{88}$$

which in turn restults in

$$\dot{M}(t) = \frac{1}{2}M_0 \veebar exp_{\veebar}(\frac{1}{2}B_b(t)) \veebar \dot{B}_b(t) = \frac{1}{2}M(t) \veebar \dot{B}_b(t) \tag{89}$$

while eqn. (86) for the bivector rate of change in the *body frame* becomes

$$\dot{M} = \frac{1}{2}M \veebar \Omega_b \tag{90}$$

using $\Omega_b = \Omega_b(t) = \dot{B}_b(t)$ as the bivector rate of change in the *body frame*. This can be transformed by multiplying it from the left by "$\underset{\sim}{M} \veebar$" to obtain

$$\Omega_b = 2\underset{\sim}{M} \veebar \dot{M} \tag{91}$$

From equations (86) and (90) we can see how world and body frame rates are connected to each other: $\Omega_w \veebar M = M \veebar \Omega_b$, which can be transformed into

$$\Omega_w = M \veebar \Omega_b \veebar \underset{\sim}{M} \tag{92a}$$

$$\Omega_b = \underset{\sim}{M} \veebar \Omega_w \veebar M \tag{92b}$$

Now we derive the rate of change of a point $X(t)$, i.e. its speed $\dot{X}(t)$, in the *world frame*, first assuming that $X(t=0) = X_0 = const.$, i.e. as a constant element that is transformed by a time dependent motor $M(t)$. During the derivation we will use the associativity of the regressive geometric product and the neutral element $\mathbb{1}$ of the regressive geometric product. We can write

$$\begin{aligned}
\dot{X}(t) &= \frac{d}{dt}X(t) = \frac{d}{dt}(M \vee X_0 \vee \underset{\sim}{M}) \\
&= \dot{M} \vee X_0 \vee \underset{\sim}{M} + M \vee X_0 \vee \underset{\sim}{\dot{M}} \\
&\overset{(83)}{=} \dot{M} \vee \mathbb{1} \vee X_0 \vee \underset{\sim}{M} - M \vee X_0 \vee (\underset{\sim}{M} \vee \dot{M} \vee \underset{\sim}{M}) \\
&\overset{(81)}{=} \dot{M} \vee (\underset{\sim}{M} \vee M) \vee X_0 \vee \underset{\sim}{M} - (M \vee X_0 \vee \underset{\sim}{M}) \vee \dot{M} \vee \underset{\sim}{M} \\
&= (\dot{M} \vee \underset{\sim}{M}) \vee (M \vee X_0 \vee \underset{\sim}{M}) - (M \vee X_0 \vee \underset{\sim}{M}) \vee (\dot{M} \vee \underset{\sim}{M})
\end{aligned}$$

From eqn. (87) we can conclude that $\frac{1}{2}\Omega_w(t) = \dot{M} \vee \underset{\sim}{M}$, resulting in

$$\begin{aligned}
\dot{X}(t) &= (\frac{1}{2}\Omega_w(t)) \vee X_w(t) - X_w(t) \vee (\frac{1}{2}\Omega_w(t)) \\
&= \frac{1}{2}(\Omega_w \vee X_w - X_w \vee \Omega_w) \\
\dot{X}_w &= rcmt(\Omega_w, X_w) = -rcmt(X_w, \Omega_w)
\end{aligned} \tag{93}$$

with $rcmt(\Omega_w, X_w)$ as the regressive commutator product between Ω_w and X_w. The resulting minus sign in eqn. (93) compared to eqn. (76) is the only difference between PGA and EGA. This is caused by the different ansatz for the motors comparing equations (58) and (62).

It can be shown that $\dot{M} \vee \underset{\sim}{M}$ is always a bivector: From equations (87) and (82), and the linearity of the involved operations, we get

$$\Omega_w = 2\dot{M} \vee \underset{\sim}{M} = -2M \vee \underset{\sim}{\dot{M}} = -2(\underset{\sim}{\dot{M} \vee \underset{\sim}{M}}) = -\underset{\sim}{\Omega}_w \tag{94}$$

Considering the grade-dependent sign-reversal of the reverse in $3D$ PGA, Ω_w must either be a vector or a bivector in PGA (see table 4 at the end of section 1.2). Since $\dot{M}$ and $\underset{\sim}{M}$ are even-grade multivectors, their geometric product results in an even-grade multivector as well, which does not have a vector part. Thus, Ω_w and therefore $\dot{M} \vee \underset{\sim}{M}$ must indeed be a bivector.

This also assures, that the grade of X and $\dot{X}$ are identical. If X is a vector then $\dot{X}$ is a vector as well. This statement is true for any grade of X as long as Ω_w is a bivector.

If X_0 is not a constant element, but a function of time, which means that X_0 is a time dependent element in the *body frame* $X_0 = X_b(t)$, we have to add another term to the derivative in eqn. (93) as follows:

$$\begin{aligned}
\dot{X}(t) &= \frac{d}{dt}(M \vee X_b \vee \underset{\sim}{M}) \\
&= \dot{M} \vee X_b \vee \underset{\sim}{M} + M \vee X_b \vee \dot{\underset{\sim}{M}} + M \vee \dot{X}_b \vee \underset{\sim}{M} \\
&= -rcmt(M \vee X_b \vee \underset{\sim}{M}, \Omega_w) + M \vee \dot{X}_b \vee \underset{\sim}{M} \\
\dot{X}_w &= -rcmt(X_w, \Omega_w) + (\dot{X}_b)_w = rcmt(\Omega_w, X_w) + (\dot{X}_b)_w \qquad (95)
\end{aligned}$$

Eqn. (95) is to be considered in the world frame. However, we can also do our derivations in a completely equivalent way using the corresponding bivector rates and rotors of the *body frame*.

The derivation in the *body frame* does not bring new insight so we don't reproduce that here for PGA again.

The final step before we move on to modelling physics is to show how points are moved in PGA using the approach described above. Every bivector B or every bivector rate Ω has its basis components

$$\Omega = (\underbrace{\boldsymbol{e}_{41}, \boldsymbol{e}_{42}, \boldsymbol{e}_{43}}_{\text{weight}(\Omega)=\Omega_\circ}, \underbrace{\boldsymbol{e}_{23}, \boldsymbol{e}_{31}, \boldsymbol{e}_{12}}_{\text{bulk}(\Omega)=\Omega_\bullet}) \tag{96}$$

where the weight part $\Omega_\circ$ carries rotational information and the bulk part $\Omega_\bullet$ carries translational information.

In general, the weight part only contains components that do contain a factor of $\boldsymbol{e}_4$. It has *directional* information on Ω. If there is zero weight, the object is contained in the horizon (H_{3dp}). The bulk part only has components that do not contain a factor of $\boldsymbol{e}_4$. It has *positional* information on Ω. If there is zero bulk, the object contains the origin (O_{3dp}).

The total bivector rate Ω can be split into

$$\Omega = \Omega_\circ + \Omega_\bullet = \Omega_{rot} + \Omega_{tra} \tag{97}$$

where eqn. (97) is called the "Euclidean split" of the bivector rate Ω because it separates the rotational and translational aspects of a moving object.

For a simple rotation around an axis $\vec{\omega} = \omega$ that contains the origin O_{3dp} and models the angular speed of rotation by the bulk norm $\|\omega\|_\bullet$ the rotational bivector rate Ω_{rot} must only contain components that contain the basis vectors $\boldsymbol{e}_{4x}$ (where $x = 1, 2, 3$). That is achieved by multiplying ω from the left with $\boldsymbol{e}_4$ using the wedge product, since it only keeps the required components.

$$\Omega_{rot} = \boldsymbol{e}_4 \wedge \omega = O_{3dp} \wedge \omega = (\omega_x \boldsymbol{e}_{41}, \omega_y \boldsymbol{e}_{42}, \omega_z \boldsymbol{e}_{43}, 0, 0, 0) \tag{98}$$

Of course, Ω_{rot} can also be expressed by a plane of rotation plr (a trivector) if the magnitude of the plane encodes the angular speed. Considering that the left weight dual of the plane $plr_\star$ provides the normal vector of the plane including its magnitude, eqn. (98) can be written in the equivalent form in terms of the plane of rotation plr

$$\Omega_{rot} = \boldsymbol{e}_4 \wedge plr_\star = O_{3dp} \wedge plr_\star \tag{99}$$

For modelling translational speed $\vec{v} = v$ we need a bivector rate Ω_{tra} that does not contain a factor of $\boldsymbol{e}_4$. This can be achieved by encoding the speed

of translation in the normal vector v of a plane and then intersecting this plane with the horizon $\overline{\boldsymbol{e}_4} = \overline{O_{3dp}} = H_{3dp}$. The plane with the corresponding attitude can be encoded by using the right bulk dual $v^\star$ to model the plane (a trivector) with the bulk norm $\|v\|_\bullet$ as magnitude. It contains the origin O_{3dp} and results in a bivector rate Ω_{tra} of the translation as

$$\Omega_{tra} = v^\star \vee \overline{\boldsymbol{e}_4} = v^\star \vee H_{3dp} = att(v^\star) = (0, 0, 0, v_x \boldsymbol{e}_{23}, v_y \boldsymbol{e}_{31}, v_z \boldsymbol{e}_{12}) \tag{100}$$

To summarize the formulas for these cases combined we can write

$$\begin{aligned} \Omega &= \Omega_{rot} + \Omega_{tra} \\ &= \boldsymbol{e}_4 \wedge \omega + v^\star \vee \overline{\boldsymbol{e}_4} \\ &= \boldsymbol{e}_4 \wedge \omega - \overline{\boldsymbol{e}_4 \wedge v} \end{aligned} \tag{101}$$

where the last transformation uses the complement rules (13a), the definition of the dual (24) as well as the fact that wedging a vector with $\boldsymbol{e}_4$ renders the need for the bulk operation unnecessary.

Using for example eqn. (93) and the Euclidean split as defined in eqn. (97) we can write

$$\begin{aligned} \dot{X}_w &= rcmt(\Omega_w, X_w) = \dot{x}_w \\ &= rcmt(\Omega_{rot} + \Omega_{tra}, X_w) \\ \dot{x}_w &= rcmt(\Omega_{rot}, X_w) + rcmt(\Omega_{tra}, X_w) && \text{(102a)} \\ \dot{x}_w &= v_{rot} + v_{tra} && \text{(102b)} \end{aligned}$$

to calculate the combined speed $\dot{x}_w$ from rotational and translational effects at a position X_w.

For a case where the rotation is around an axis that does not include the origin O_{3dp}, but another reference point P_{ref} on the rotational axis, use $X_w - P_{ref}$ as argument of the regressive commutator product for the rotational part instead of X_w. This transforms the reference point as new fictive origin to obtain the correct rotational speed around the reference point. Everything else can remain unchanged.

1.4 Physics modelling

The key idea is to represent physical entities using objects that are represented directly in the underlying algebra. They have a geometric interpretation and can be directly manipulated by applying algebraic operations.

EGA is limited to objects that contain the origin. Thus, it can be used to directly model vectors (i.e. directions), lines, and planes through the orgin. In addition, it can model complex numbers, quaternions and thus rotations in $2D$ and $3D$ well. To realize them, it uses sandwich products based on the geometric product for calculating the transformation resulting from two consequtive reflections either across lines ($2d$-case) or across planes ($3D$-case). The resulting transformations are rotations. The advantage of using geometric algebra to model those rotations is the intuitive modelling, due to geometric meaning of the objects and the fact that the sandwich products can be used to transform different objects, like vectors, bivectors or rotation operators using the same sandwich product formulas in a unified fashion regardless of the type of object to be transformed.

In addition, PGA is capable to model objects that do not necessarily contain the origin. Thus it can be used to directly model points, directions, lines, and planes that may or may not contain the origin. Translations can be implemented in PGA directly as orthogonal transformation using a sandwich product. The operation is realized by two consequtive reflections across parallel lines ($2D$) or planes ($3D$). It is based on the regressive form of the geometric product. The reason for this deviation from EGA, which uses the geometric product for that purpose, is that the Euclidean isometries (rotation, translation, and screw motion) can only be performed in a space that is dual to the space we use for modelling our projective objects (for explanation see [Lengyel, 2024b]). The use of the regressive geometric product on these objects is equivalent to first transforming the object into dual space by applying the complement operation, then using the geometric product in that space for the intended transformation and finally transforming the resulting transformed object back into the original space, by applying the inverse complement operation. Thus the sandwich product transformation formulas in PGA differ from the formulas used in EGA by using the regressive geometric product instead of the geometric product, but are otherwise the same.

The additional degree of freedom in PGA is possible due to embedding the Euclidean space of dimension n into a projective space of dimension $n+1$, e.g. two-dimensional Euclidean space is modelled by embedding it in a three-

dimensional projective space, and three-dimensional Euclidean space is modelled in a four-dimensional projective space. In the $n+1$-dimensional projective space all algebraic objects contain the origin (exactly as in EGA), but in the modelled n-dimensional Euclidean space the objects do not need to contain the origin. They can be located anywhere in the modelled space including postions at infinity. In projective geometry the objects must be projected into the modelled space. Thus the objects of the embedding space have one dimension more than the corresponding objects in modelled space. A projective point (which models a scalar or $0d$-object) is a vector representing a ray in the embedding space (a $1d$-object), and (a $1d$-object) like a line in modelled space is created by a bivector in the embedding projective space (a $2d$-object), and so on for higher grade objects. The projection reduces the dimension of the object in the embedding space by one.

Due to its extended expressiveness PGA is suitable to model physics of mass points and rigid bodies. For example it can model position, velocity, acceleration, force, torque, momentum, angular momentum, etc., and thus it can be used to describe kinematics and dynamics of mass points and rigid bodies. The approach used here is inspired by [Dorst and De Keninck, 2022] and [Dorst and De Keninck, 2023], but it does not use the plane-based ansatz with planes as vectors, which are based on dual representations of the objects to be modelled. It rather uses the approach as described by [Lengyel, 2024b] using projective geometric algebra based on direct representations of points, lines and planes, who also gives reasons for choosing the latter approach and explains the background in more detail compared to the short indications given here.

Objects modelled in projective space can be multiplied by a scalar value without changing the *geometric* meaninig of the object in the modelled Euclidean space, i.e. its interpretation after the projection. Equality of modelled geometric objects is thus often only defined up to a scalar factor. This kind of equality up to a scalar factor is defined as congruence. Therefore, it is important for modelling of physical facts that unitization is used consciously to avoid false interpretations. For example, a heavy point can be modelled by multiplying a *unitized* point with the scalar value of its mass, but of course the multiplication with m must happen after unitization in order to keep the scalar factor m in the calculation. Other geometric objects need to be unitized in order to focus on the intended geometric operation and avoid additional scaling, i.e. the rotation operators. They must be unitized before use as well. Thus it is advisable to think through the geometric operations and their physical interpretation well to avoid suprises and to be consistent.

[Lengyel, 2024a] points out potential mathematical inconsistencies in applications of geometric algebra as often used up-to-now. Further hints on inconsistencies in some GA approaches can be found as well in [Kritchevsky, 2024]. The goal of the this work is to be as consistent and mathematically rigoros as possible implementing GA functionality, and at the same time providing an intuitively applicable framework for modelling of physics and applications.

1.4.1 Modelling force and torque

[Browne, 2012] (see sections 4.1-4.5) is a very good source showing how to use Grassmann algebra and projective geometry in order to model forces and moments. Classical approaches based-on vector algebra only model forces by providing vectors of adequate magnitude and direction to represent the force. However, the physically very meaningful "line of action" is not modelled mathematically at all in the classical approach. It rather needs to be derived from the description of the context in order to know where the force is actually acting.

This changes in PGA. Any force can be directly modelled by its line of action which encodes the force *and* the torque generated by the force at the origin within one object. A force f is modelled by a line F. If f is the force vector of corresponding magnitude and direction, and Q is any unitized projective point on its line of action then F is the resulting line of action:

$$F = Q \wedge f \tag{103}$$

The resulting bivector F represents a line, a geometric object which contains all physically relevant information of the force f and the torque generated by f. For that reason [Dorst and De Keninck, 2023] call this object "forque". It can directly be used to recover the force vector and to calculate the resulting free torque by the force either at the origin or at any point in space as shown below.

But first let's have a closer look how this is accomplished. We can separate F into its components. Each unitized projective point Q is the sum of the origin O and a vector q parallel to the projective plane pointing from O to Q, such that $Q = O + q$ ("projective point split"). Thus F can be split into two bivector components

$$F = Q \wedge f = (O + q) \wedge f = \underbrace{O \wedge f}_{=\text{force}} + \underbrace{q \wedge f}_{=torque} = F_\circ + F_\bullet \tag{104}$$

where the first term encodes the force as within the weight components of the bivector, while the second term encodes the resulting free torque relative to the origin caused be the action of f at Q – or alternatively anywhere along the line of action – within the bulk components of the bivector.

To see this even more clearly, let's look at the component equations of both terms written as resulting bivectors for the case of $2D\ PGA$ (the corresponding basis bivector compontents are shown on the right-hand side in square brackets for reference):

$$O \wedge f = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \wedge \begin{pmatrix} f_x \\ f_y \\ 0 \end{pmatrix} = \begin{pmatrix} -f_y \\ f_x \\ 0 \end{pmatrix} \qquad \begin{bmatrix} \boldsymbol{e}_{23} \\ \boldsymbol{e}_{31} \\ \boldsymbol{e}_{12} \end{bmatrix} \tag{105a}$$

$$q \wedge f = \begin{pmatrix} q_x \\ q_y \\ 0 \end{pmatrix} \wedge \begin{pmatrix} f_x \\ f_y \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ q_x f_y - q_y f_x \end{pmatrix} \qquad \begin{bmatrix} \boldsymbol{e}_{23} \\ \boldsymbol{e}_{31} \\ \boldsymbol{e}_{12} \end{bmatrix} \tag{105b}$$

where equation (105a) shows how the force is encoded in the first two components of the bivector and equation (105b) shows how the torque is encoded in the third component of the bivector F.

For $3D\ PGA$ the encoding is similar, but of course now requires six bivector components instead of three (the corresponding basis bivector compontents are shown on the right-hand side in square brackets for reference):

$$O \wedge f = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} \wedge \begin{pmatrix} f_x \\ f_y \\ f_z \\ 0 \end{pmatrix} = \begin{pmatrix} f_x \\ f_y \\ f_z \\ 0 \\ 0 \\ 0 \end{pmatrix} \qquad \begin{bmatrix} \boldsymbol{e}_{41} \\ \boldsymbol{e}_{42} \\ \boldsymbol{e}_{43} \\ \boldsymbol{e}_{23} \\ \boldsymbol{e}_{31} \\ \boldsymbol{e}_{12} \end{bmatrix} \tag{106a}$$

$$q \wedge f = \begin{pmatrix} q_x \\ q_y \\ q_z \\ 0 \end{pmatrix} \wedge \begin{pmatrix} f_x \\ f_y \\ f_z \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ q_y f_z - q_z f_y \\ q_z f_x - q_x f_z \\ q_x f_y - q_y f_x \end{pmatrix} \qquad \begin{bmatrix} \boldsymbol{e}_{41} \\ \boldsymbol{e}_{42} \\ \boldsymbol{e}_{43} \\ \boldsymbol{e}_{23} \\ \boldsymbol{e}_{31} \\ \boldsymbol{e}_{12} \end{bmatrix} \tag{106b}$$

The force vector f itself can be retrieved from F by using the attitude function $att()$:

$$f = att(F) = F \vee \overline{\boldsymbol{e}_n} \tag{107}$$

with $\boldsymbol{e}_n$ denoting the basis vector of the projective dimension that squares to zero in the respective algebra. The torque generated relative to the origin O is encoded in the bulk of the bivector:

$$M_O = bulk(F) = F_\bullet \tag{108}$$

while the resulting torque from the application of f at any given point R in space can be retrieved from F, either using any known point Q on F[3], or alternatively by projecting R onto F and using the perpendicular distance vector $d_\perp$ from R towards the line F in order to calculate the resulting free torque $M_F(R)$ as

$$M_F(R) = [(Q - R) \wedge f]_\bullet = [(Q - R) \wedge att(F)]_\bullet \tag{109a}$$

$$M_F(R) = [d_\perp \wedge f]_\bullet = [(F \vee (R \wedge F^*) - R) \wedge att(F)]_\bullet \tag{109b}$$

where equation (109a) is the formula with the least computational effort using a known point Q on F, while equation (109b) uses the othogonal projection of R onto F to calculate the perpendicular distance vector $d_\perp$.

Please remember:

- Neither a vector nor a bivector are located anywhere. They can be moved around freely. A line however represents a bound vector that is bound by its line. Thus the vector can only be moved along its line without changing the geometric and physical meaning (see [Browne, 2012], p. 169).
- A free torque is modelled by a bivector with force components being zero, i.e. the force vector has no direction and zero length. But the bivector still will contain a resulting torque value.
- The torque calculated from either equation (109a) or (109b) is necessarily a free torque as can be seen from the formulas. The origin does not occur in the result anymore due to the difference of the two projective points involved.

[3] Which is always available via the function $support(F)$. It provides the point on F with the shortest distance to the origin O by orthogonally projecting the origin onto F.

- The sum of forces and their induced torque values can be directly created by the sum of their respective lines of action, i.e. by summing the bivectors representing those lines. The resulting lines of action contain all relevant information incl. the resulting vector sum of the forces and the resulting torque values from the summation. There is no need to calculate these separately. The result will always be a resulting bivector that potentially contains a bound vector and a resulting torque. (see [Browne, 2012], p. 176).

1.4.2 Modelling linear and angular momentum

Momentum in the classical approach is split into linear momentum $\vec{p} = m\vec{v}$ and angular momentum $\vec{L} = \vec{x} \times \vec{p} = \vec{x} \times m\vec{v}$. Both are represented as vectors, but they represent entities with different physical meaning. So they cannot be combined in one object in the classical approach.

The PGA approach to momentum is to define both the linear and the angular aspects within one entity. This is in complete analogy to what we have seen in the last subsection for force and torque resulting in a force line of action.

Projective momentum P of a point mass m at position X that moves with velocity $\dot{X}$ is defined as a bivector P where

$$P = X \wedge m\dot{X} \tag{110}$$

It defines the overall momentum at position X including linear and angular momentum in the bivector $P(X)$, which can be shown by splitting the projective point X into its components $X = O + x$ ("projective point split"). In the $3D$-case with the origin $O = O_{3dp} = (0, 0, 0, 1)$ and the direction vector x, where $x = (X_x, X_y, X_z, 0)$, i.e. specifically with $X_w = 0$, which represents a direction towards a point at infinity, this results in

$$\begin{aligned}
P(X) &= X \wedge m\dot{X} = (O + x) \wedge m\dot{X} \\
&= O \wedge m\dot{X} + x \wedge m\dot{X} \\
&= O \wedge m(\dot{O + x}) + x \wedge m(\dot{O + x}) \\
&= O \wedge m\dot{x} + x \wedge m\dot{x} && \text{(111a)} \\
&= \underbrace{P_\circ}_{\text{linear}} + \underbrace{P_\bullet}_{\text{angular}} && \text{(111b)}
\end{aligned}$$

P defines an instantaneous momentum line where the weight part $P_\circ$ of the bivector P encodes the linear momentum and the bulk part $P_\bullet$ encodes the

angular momentum with respect to the origin.[4]

The physical aspects encoded in the momentum line P can be retrieved in complete analogy to the case of force and torque in subsection 1.4.1:

The linear momentum vector p itself can be retrieved from P by using the attitude function $att()$:

$$p = mv = m\dot{x} = att(P) = P \vee \overline{\boldsymbol{e}_n} \tag{112}$$

with $\boldsymbol{e}_n$ denoting the basis vector of the projective dimension that squares to zero in the respective algebra. The angular momentum L generated relative to the origin O is encoded in the bulk of the bivector:

$$L_O = x \wedge m\dot{x} = bulk(P) = P_\bullet \tag{113}$$

while the resulting angular momentum relative to any given point R in space can be retrieved from the momentum line P by

$$L(R) = \left[(X - R) \wedge m\dot{X}\right]_\bullet = \left[(X - R) \wedge att(P)\right]_\bullet \tag{114}$$

using X as a known point on P. This provides the resulting angular momentum $L(R)$ with respect to R. The momentum line encodes the resulting momentum and its linear and angular aspects relative to any arbitrarily chosen point.

The equations (112) and (113) given above permit to separate the linear and angular effects as in the classical approach. However, these aspects need not be separated at all and can be calculated together in PGA.

Using eqn. (110) and the definition $X = O + x$ with X as a unitized projective point, O as the chosen origin and x as the direction vector pointing from O to X we can write

$$P_O(X) = X \wedge m\dot{X} = (O + x) \wedge m(O \dot{+} x) \tag{115}$$

Compared to eqn. (110) an index O was added to P, since the formula (115) seems to confirm that the momentum depends on the choice of the origin.

[4]The "projective point split" works in this way, since the origin is a destinguishable point. This is in contrast to EGA or the classical approach where the orgin is a null-vector, which has no lasting effect if it is added to another point. This is one reason why PGA can combine linear and angular aspects as separate physical effects in a single algebraic object. The other is that the required 6 degrees of freedom can be encoded in a bivector.

Now it is important to demonstrate that this is *not* the case. We are going to prove that the described physics does not depend on an arbitrarily chosen origin and not on the resulting coordinates used for its description.

First approach: Use a transformation of the standpoint from which we describe the physical situation without actually moving the coordinate system (we still want use the chosen coordinate system to describe the vectors and bivectors used): We go to a virtual origin $O' = Y$, but describe the situation in the same coordinate system. This results in describing the origin as $O = Y - y$ as seen from Y. Inserting that definition into eqn. (115) and using the geometric vector identity

$$X = Y + (X - Y) = Y + (O + x - (O + y)) = Y + (x - y) \tag{116}$$

we express $P(X)$ as viewed from Y by reformulating the equation for P relative to the origin O

$$\begin{aligned} P_O(X) &= X \wedge m\dot{X} = (O + x) \wedge m\dot{(O + x)} \\ &= (Y - y + x) \wedge m\dot{(Y - y + x)} \\ &= (Y + (x - y)) \wedge m\dot{(Y + (x - y))} = P_Y(X) \end{aligned} \tag{117a}$$

where the last equation obviously is the same as eqn. (115), but here Y takes the role of O and $(x - y)$ takes the role of the vector x in the old description. So the same physical situation was described from another viewpoint. However, the algebraic object P is unchanged, i.e. $P_O(X) = P_Y(X)$. This clearly shows that P does not depend on the chosen origin and that eqn. (115) is indeed a coordinate free definition.

Second approach: This one is not so explicit on being independent of the origin. We just express X by Y using eqn. (115) and (116) directly to write

$$\begin{aligned} P_O(X) &= X \wedge m\dot{X} \\ &= (Y + (X - Y)) \wedge m\dot{(Y + (X - Y))} \\ &= (Y + (x - y)) \wedge m\dot{(Y + (x - y))} = P_Y(X) \end{aligned} \tag{118a}$$

We can again see that this is structurally identical to eqn. (115) just seen from Y instead from O.

The total momentum of a set of mass points $m_i X_i$ is defined as the sum of all momentum blades.

$$P_{total} = \sum_i P_i \tag{119}$$

P_{total} will always be a momentum blade in $2D\ PGA$, while in $3D\ PGA$ (a four-dimensional algebra) it could also result in a general bivector that might not be factorizable into a blade (see [Browne, 2012]). That would be the case, if $P \wedge P \neq 0\ \mathbb{1}$, i.e. when the result is a pseudoscalar that is not equal to a zero multiple of the pseudoscalar $\boldsymbol{e}_{1234}$. This can always be used to determine whether a bivector is a blade in $3D\ PGA$.

1.5 Glossary

Moment and torque (source wikipedia):

- A moment is a mathematical expression involving the product of a distance and a physical quantity such as a force or electric charge. Moments are usually defined with respect to a fixed reference point and refer to physical quantities located some distance from the reference point. For example, the moment of force, often called torque, is the product of a force on an object and the distance from the reference point to the object. In principle, any physical quantity can be multiplied by a distance to produce a moment.

- In physics and mechanics, torque is the rotational correspondent of linear force. It is also referred to as the moment of force, or simply the moment. Just as a linear force is a push or a pull applied to a body, a torque can be thought of as a twist applied to an object with respect to a chosen point.

- Torque is generally referred to using different vocabulary depending on geographical location and field of study. Torque and moment are often used interchangeably for the rotational effect of a force, but in mechanics, the main difference is application: torque refers to a force causing rotation in a dynamic system, while moment is often used for the turning effect in a static, or non-moving, system. Both are calculated as the force multiplied by the perpendicular distance from the pivot point.

- When modelling physics in PGA forces and moments, describing the translational and rotational physical aspects, do not need to be considered separately, but can be modelled within one object containing all information. For example, by modelling the line of action F of a force f acting at a point P as $F = P \wedge f$ as in equation (103) both effects are covered, the force f itself and the torque generated at any point are covered by the mathematical object F. The torque generated at the origin $M_O = bulk(F)$ can be as easily recovered from F as the resulting free moment at any other point.

1.6 Literature

References

[Browne, 2012] Browne, J. (2012). *Grassmann Algebra*, volume 1: Foundations. Barnard Publishing.

[Dorst and De Keninck, 2023] Dorst, L. and De Keninck, S. (August 15, 2023). May the forque be with you - dynamics in pga. *University of Amsterdam, The Netherlands*, V2.6.

[Dorst and De Keninck, 2022] Dorst, L. and De Keninck, S. (March 14, 2022). A guided tour to the plane-based geometric algebra pga. *University of Amsterdam, The Netherlands*, V2.0.

[Gunn, 2020] Gunn, C. G. (August 2020). Projective geometric algebra: A new framework for doing euclidean geometry. *Raum+Gegenraum Falkensee, Germany*.

[Kritchevsky, 2024] Kritchevsky, A. (February 28, 2024). The case against geometric algebra. *Alex Kritchevsky's blog*.

[Lengyel, 2024b] Lengyel, E. (2024b). *Projective Geometric Algebra Illuminated*. Terathon Software LLC.

[Lengyel, 2024a] Lengyel, E. (August 23, 2024a). Poor Foundations in Geometric Algebra. *Eric Lengyel's blog*.

[Lengyel, 2025] Lengyel, E. (May 23, 2025). The Transwedge Product. *Eric Lengyel's blog*.